

WHERE THE SECONDS GO: BOUNDING MIXTURE-OF-EXPERTS DECODE FROM HOST MEMORY, AND MEASURING WHAT FORESIGHT IS WORTH

Anonymous Authors¹

ABSTRACT

A Mixture-of-Experts model too large for one consumer GPU keeps most of its experts in host memory, and decoding one token at a time then waits on host reads. We bound how fast such decoding can be on a given machine, measure where the remaining time goes, and price knowledge of the future routing. The bound is the host reads of Belady’s MIN with bypass, the fewest that any cache holding C experts per layer can make, at the machine’s probed read rate. Our llama.cpp expert cache, faster than two other systems at most configurations, reaches 31–54% of the bound’s speed on machines with consumer processors at the smallest budget we test. Oracles inside the engine account for the rest of the time: caching the experts MIN would keep pays only if each admitted expert is also read once instead of twice, and the two changes together close 35% of the gap in a test registered before its machines were rented. Where the link reads at least half as fast as the CPU, policies without foresight recover at most 6% of the oracles’ gain. Half of foresight’s value needs the routing of about $0.65 C$ distinct experts per layer ahead.

1 INTRODUCTION

Open-weight Mixture-of-Experts (MoE) models now carry 30–120 B parameters and activate a few billion per token; a desktop GPU has 24–32 GB. Running such a model for one user therefore means keeping most experts in host DRAM. Each decoded token reads its routed experts from GPU memory if they are resident, and otherwise from host memory: either the CPU runs the expert, or the expert is copied over PCIe into a GPU slot (Ivanov et al., 2021; Hoefler et al., 2021).

Many systems build on this: GPU expert caches (Eliseev & Mazur, 2023; Xue et al., 2024; Song et al., 2024a), CPU execution of misses (Kamahori et al., 2024; Chen et al., 2025a; Zhong et al., 2025), a per-miss choice between the two (Yang et al., 2026; Zhu et al., 2026) and expert prediction (Hwang et al., 2024; Zhu et al., 2025; Wang et al., 2026b). Each reports a speed-up over a baseline of its choosing. None says how fast the same machine could decode the model, or which part of the remaining time a better design could remove. Without that, a builder cannot tell whether to work on what is cached, how it is read, when it is read, or how far ahead the routing must be predicted.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

We answer these questions for batch-1 decode of gpt-oss-120b (OpenAI, 2025) and Qwen3-30B-A3B (Yang et al., 2025) on rented RTX 5090 machines. We derive a bound, build oracles into a real engine that remove one part of the gap at a time, and run them on many machines, with every prediction committed before its machine started. Table 1 lists the answers and the evidence behind each.

The headroom is large (Section 3). MIN’s host reads at the machine’s probed rate bound the time per token of any system with C slots per layer, and that read time is nearly reachable in a microbenchmark. Our cache, which leads FreeToken and llama.cpp at most configurations, runs at 31–54% of the bound’s speed at the smallest gpt-oss budget on machines with consumer processors; published systems, scored the same way, reach a median 13.6%.

What is cached and how it is read pay only together (Section 4). At the smallest gpt-oss budget, caching MIN’s set on the deployed read path, where the CPU serves a miss and a background copy fills its slot, closes almost none of the gap; fetching the deployed set into its slots in the step closes none either. Doing both closes 33% of the gap on 15 panel machines, and 35% on 5 machines rented later for a registered test. Landing the copies in time, in a registered control, does not rescue the two-read path. What the oracles leave we attribute to the GPU’s own work in series with the reads and to the oracles’ extra reads (Table 4).

Without foresight, little of it is reachable (Section 6.1). Four variants of the deployed cache built from the engine’s

admission margin and its layer-ahead prediction recover at most 6% of the read-ahead oracle’s gain on those machines: they still read as many experts as the deployed cache.

The machine decides how foresight should be spent (Sections 5 and 6.2). Each admitted expert should be read once, with as few admissions as optimality allows, on the read path the link-to-CPU ratio favours. A trend in that ratio, fitted on RTX 5090 machines, predicted the oracles’ speed on five RTX 4090 machines at ratios 0.37–0.77 within 7%; new RTX 5090 machines with fast links scatter more around it.

Foresight has a price (Section 7). Half of its value needs the routing of about $0.65 C$ distinct experts per layer ahead, on 9 models; exact routing of the next token recovers at most a third of it on our workload.

Section 8 turns these results into rules for system builders.

Contributions.

- **A bound on time per token** for exact-routing systems with C slots per layer, shown to be nearly reachable in its read time, and the distance of our cache and of published systems from it (Section 3).
- **A measured decomposition of the gap** with oracles inside a real engine, on 15 machines and replicated on 5 more under a registered test (Section 4).
- **How foresight must be spent:** read once, with the fewest admissions, on the path the machine favours (Section 5); without foresight, the engine’s own mechanisms recover little, and a trend in the link-to-CPU ratio predicted a second card’s oracles (Section 6).
- **A price on foresight** by horizon and accuracy, and a horizon rule in distinct experts across 9 models (Section 7).
- **An open record:** every prediction committed before the machine it concerns started, including the ones that failed, and every number generated by a script (Appendices D and L).

2 SETTING

Models and workload. gpt-oss-120b has 36 MoE layers of 128 experts, top-4, in MXFP4 (13.25 MB per expert). Qwen3-30B-A3B has 48 layers of 128 experts, top-8, in BF16 (9.44 MB per expert). Prompts are AIME-25 problems, and the cache carries over between problems, as in one user’s session. The system comparison decodes 256 tokens of each of the 30 problems through each system’s server. The engine experiments replay the model’s own greedy output, teacher-forced, so that every configuration sees the same routing. A *budget* is the share C/E of each

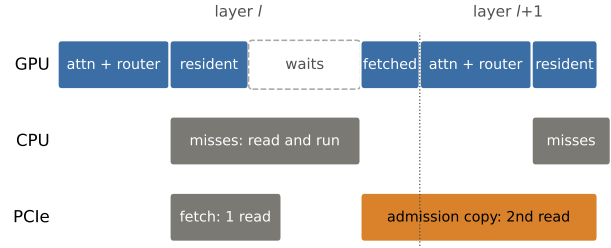


Figure 1. One decode step of the deployed cache, schematically. Blue: the GPU’s own kernels. Grey: host reads of the misses, which the step waits for. Orange: the background copy that fills an admitted expert’s slot, a second read of the same expert.

layer’s experts that fit in GPU slots: gpt-oss 11% and 25% are $C=14$ and $C=32$ of 128. The two smallest budgets of each model are *host-bound*: at the GPU’s datasheet rate, host reads, not the GPU, limit their speed in Eq. (1), except at gpt-oss 25% on the few machines with the fastest host memory. At the GPU’s measured batch-1 rate the GPU term also binds at gpt-oss 25% on both hosts of Table 3, and at Qwen3 25% on host B. Most results are at gpt-oss 11% and 25%.

The engine. Our cache is a 3,500-line patch to llama.cpp (Gerganov & contributors, 2023–2026). Each MoE layer has C GPU slots. A *miss* is read from host memory in one of two ways: CPU helper threads run it, or it is *fetched* (copied into a slot over PCIe and run on the GPU in the same step). A per-machine table, computed from the machine’s bandwidth probe, sets how many of a layer’s misses are fetched. The *deployed* policy admits a miss into a slot when its decayed request count beats the lowest resident’s by a margin $\kappa=1$. The CPU serves the admitted miss and a background copy then fills its slot, so the expert is read from host memory twice (Fig. 1).

Terms. *MIN* is Belady’s optimum with bypass (Belady, 1966; Zhang et al., 2020): it *admits* a miss only if some resident is needed later than the miss, which gives the most hits possible with C slots (Carlisle & Lloyd, 1995; Jain & Lin, 2016). Many schedules reach that optimum. The *greedy* one admits whenever it may; the *fewest-admission* one admits as rarely as any of them. An admitted expert is read *once* (1R), fetched into its slot in the step that needs it, or *twice* (2R), served by the CPU in that step and copied in the background; the engine publishes a background copy at the start of the second step after the one that admitted it. A copy issued a few steps before the expert’s use is read *ahead*. *Oracles* read the future routing from a trace recorded on the same machine. Configurations are named by set and reads: MIN-1R is MIN’s greedy set read once, MIN-2R the same

set read twice, Dep-1R the deployed set read once; Table 2 lists every configuration, and Appendix C maps the codes of the appendix tables to these names.

Machines. Every machine is a rented host with one RTX 5090 (an RTX 4090 in Section 6.2). Each runs our bandwidth probe before any model run. It reads host memory from CPU threads, at the number of helper threads the engine uses (B_c), and over PCIe into the GPU, at the rate the engine’s copies use (B_p); B_{host} is its highest reading by any method. The *link-to-CPU ratio* is B_p/B_c . We call a link *fast* when the ratio is at least 0.5 and *slow* otherwise; the term compares the link with the CPU, not with other links, and we drew the 0.5 line after the panel’s data, before the registered tests of Section 6. Ryzen, Core and Threadripper parts, high-end desktop ones included, are *consumer* processors; EPYC, Xeon and engineering-sample parts are *server* processors.

Statistics and registration. Time per token is averaged over problems, and speed ratios are ratios of mean times. Machines differ far more than problems: two panel machines with the same CPU model differ by up to 29% in the deployed cache’s time, while machines rented again came within 3.2% (RTX 5090) and 5.5% (RTX 4090) of their earlier runs. The machine is therefore the unit. Claims are about orderings that hold on every machine of a stated class, and summaries over machines carry 95% t -intervals over machines, since with a handful of machines a percentile bootstrap over machines is too narrow (Kalibera & Jones, 2013); comparisons on one machine carry 95% intervals over problems (a paired bootstrap). A *launch* is one rental; in the later experiments each configuration also ran in two or three separate processes per launch. A machine *ran stably* when its outputs matched the reference and, where it ran more than one process, their times agreed within 2%; two experiments ran each configuration in a single process, so only the first condition applies to them (Appendix D). Every experiment’s predictions were committed to a public branch before its machine started; Appendix D scores all of them.

Yardsticks. The common one is speed relative to the deployed cache on the same machine (“1.22 $\times$ ”). Three others are shares: Section 3 gives a system’s speed as a share of the speed of Eq. (1), *the bound*; Section 4 splits the *gap*, the deployed cache’s time beyond Eq. (1), into shares; and Section 6 gives a policy’s *capture*, its share of the read-ahead oracle’s gain over the deployed cache. Section 7 also gives a window’s share of what full foresight saves over admitting every miss, and says when.

3 HOW FAST COULD IT BE?

Consider any system that executes the model’s exact routing with at most C experts of each layer resident on the GPU. Per token, each of the Lk routed experts is served by a resident copy or read from host memory. Let R^* be MIN’s host reads per token, summed over layers; no policy, even one that knows the future, reads fewer. If m experts per token run on the CPU, host memory supplies at least $\max(R^*, m)S$ bytes for experts of S bytes, and the GPU reads the dense weights D plus $(Lk - m)S$. With everything overlapped,

$$\bar{T} \geq \min_{0 \leq m \leq Lk} \max \left(\frac{D + (Lk - m)S}{B_{\text{gpu}}}, \frac{\max(R^*, m)S}{B_{\text{host}}} \right). \quad (1)$$

At a host-bound budget this is MIN’s read time, R^*S/B_{host} . The bound assumes exact routing, whole experts, at most C residents per layer and one token per step. It drops all latencies, lets the two read paths share B_{host} , and is relative to the probe: a reader faster than the probe could beat it (Section 10).

The demand bound: reads that wait for the router. A system that reads an expert only after its layer’s router has chosen it cannot overlap that read with the GPU’s work: layer l ’s misses are known only after its attention and router, and layer $l+1$ starts only after layer l ’s experts. Its time is at least the GPU’s non-expert work plus the reads, in series:

$$\bar{T}_{\text{demand}} \geq T_{\text{GPU}} + \frac{R^*S}{B_{\text{host}}}, \quad (2)$$

where T_{GPU} is the GPU’s attention, dense layers, norms and routers per token: 2.9 ms for gpt-oss, the smallest of our engine’s Nsight profiles; a system with faster non-expert kernels has a lower bound. Every system we compare reads on demand; only reads issued before routing can go below Eq. (2).

The ordered bound: reads foresight cannot free. Foresight does not free every read. An expert run on the CPU needs its layer’s attention output, and the next layer’s attention needs its result, so CPU reads sit in series with the GPU’s non-expert work; only copies over the link can be issued ahead. This assumes that the CPU reads an expert’s weights from memory when it runs it; a read-ahead into the CPU’s cache would relax it. With a share f of MIN’s reads served by the CPU at the probe’s highest CPU reading $\hat{B}_c$ and the rest copied at its highest link reading $\hat{B}_p$,

$$\bar{T} \geq \min_{0 \leq f \leq 1} \max \left(\frac{(1-f)R^*S}{\hat{B}_p}, T_{\text{GPU}} + \frac{fR^*S}{\hat{B}_c} \right), \quad (3)$$

and Eq. (1) holds as well, so the bound is the larger of the two. On the median consumer machine at gpt-oss 11%,

Eq. (3) equals Eq. (1) (ratio 1.00); where the link is slow, it is up to 1.56 times as high.

The read time is nearly reachable (registered at 50% or more per layer). A microbenchmark replays MIN’s per-layer reads of gpt-oss on 3 machines without the model: CPU threads read part of each layer’s experts while the copy engine copies the rest. It reaches 86–96% of the bound’s read time (Table 29). The GPU term of Eq. (1) is looser, but at datasheet rates it binds only at the largest budgets (Table 28).

Where systems stand. Table 3 places three systems against the bounds on two machines. Our cache leads Free-Token (Yang et al., 2026) at 11 of 12 configurations and stock llama.cpp at all of them (Appendix J). Across 25 machines with consumer processors at gpt-oss 11%, it runs at 31–54% of the speed of Eq. (1), the bound we use throughout (40–54% of Eq. (3)); the 5 machines added in Section 4 fall at 41–48%. The top of that range is probably too high: on that machine the engine’s reads imply, by the closed-form account of Appendix H, a rate above the probe’s best reading (Table 27).

The field leaves most of the machine unused (post hoc). To compare with published systems, we score each one against the same bound computed from its paper’s own hardware at datasheet rates, not probed ones. Over 20 published batch-1 measurements, the median is 13.6% of the bound; our cache, scored the same way on our own hardware, reaches a median 27% (Appendix K). Three caveats apply. Where we lack a model’s routing trace, uniform routing stands in, which overstates the bound’s reads and so the published percentages. The bound is per layer and one token per step, so designs that pool their cache across layers or verify several tokens per pass are scored against a bound they could in principle beat. And the comparison with our cache crosses hardware.

4 WHERE THE SECONDS GO

The gap is the deployed cache’s time beyond Eq. (1). Oracles in the engine remove three parts of it: what is cached (MIN’s set instead of the deployed one), how it is read (each admitted expert fetched once in the step, instead of served by the CPU and copied in the background), and when (the read-ahead oracle issues its copies a few steps early). We change the first two alone and together, on the 15 panel machines that ran every state and, under a test registered before they started, on 5 machines rented for it (Fig. 2 and Table 4).

What is cached and how it is read pay only together (registered: a positive interaction on every machine). At gpt-oss 11%, MIN-2R, MIN’s set on the deployed read

path, closes 0% of the gap, and Dep-1R, the deployed set read once, closes 0%. MIN-1R, both changes, closes 33% on the panel and 35% on the new machines. MIN’s set admits more experts than the deployed one, and on the deployed path each extra admission costs a second read. Its copies also land two steps after the admission, so MIN-2R misses more than MIN-1R: the factor is the read path, which sets both the number of reads and when a copy lands. A registered control separates the two (Table 5). Issuing each copy of MIN’s fewest-admission set one step earlier, so that it lands for the next use, does not rescue the two-read arm; it makes it slightly slower. On four machines (both cards, ratios 0.57–0.88) at 11%, Few-2R-early runs $0.98\text{--}1.00\times$ the deployed cache and Few-2R $1.03\times$, against $1.21\text{--}1.41\times$ for the same set read once (Few-1R). Its misses barely fall (0.97–0.98 of Few-2R’s, where we had registered at most 0.95 from a replay) and stay above Few-1R’s by 12.1–13.8 per token. The counters point to the path itself: on it the machine’s fetch table still fetches 17.8–23.7 of each token’s 53.0–54.6 misses in the step, into slots the schedule did not choose, so MIN’s set is not kept even when its copies land in time. The late landing is not what holds the two-read arms back. We registered the sign of this interaction on the 10 machines of the panel’s main job (it held on 9) and again on the new machines (it held on 5 of 5). At 25%, where admissions are fewer, MIN’s set alone already closes 21%.

What is left. The read-ahead oracle closes another 15% at 11%, and 52% of the gap remains (Table 4). On fast links, two named parts account for it. The oracle still reads its remaining misses after their router runs, so the GPU’s non-expert time stays in series with them, and it reads more than MIN, because its early copies evict sooner. A system that issued every read before routing and read no more than MIN would remove both, but only copies over the link can be issued early (Eq. (3)). On slow links the two parts fall short: a residual of 35–47% of the gap at 11% stays unexplained on the two slow-link panel machines, and 31–36% on the three slow-link RTX 4090s of Section 6.2. Measuring the gap to Eq. (3), the tighter bound there, does not remove it: the read-ahead oracle leaves 83–99% of that gap on the two machines and 73–81% on the three RTX 4090s, against a median of 45% on fast links. The oracles fetch their misses over the link, which is the slow path on these machines, so their residual there reflects the read path they use.

A closed-form account (exploratory, not confirmed). On consumer processors, the deployed cache’s time is close to the GPU’s profiled kernel time plus its counted host reads at B_{host} . We registered this, first in a plainer form, before four experiments: three failed and one was inconclusive by its own rule (Appendix H). The measured decomposition does not rely on it.

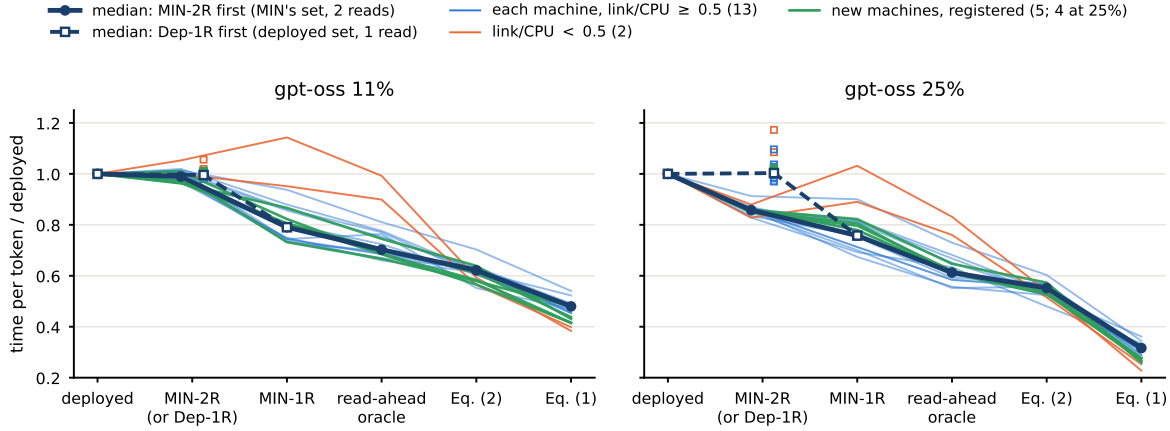


Figure 2. Where the seconds go, measured: each state’s time per token relative to the deployed cache on the same machine (lower is faster), as oracles change what is cached and how it is read (alone, then together) and when it is read, with the two bounds for reference. One line per machine. Blue and orange: the panel, split at a link-to-CPU ratio of 0.5; on the two slow-link machines (orange), reading once in the step saves little or costs time. Green: the machines of the registered test. Thick dark lines: medians over the fast-link panel machines, changing MIN’s set first (solid) or the number of reads first (dashed).

5 HOW FORESIGHT MUST BE SPENT

This section measures each way of spending foresight as a speed relative to the deployed cache on the same machine. A running example: on a Ryzen 9 9950X at gpt-oss 11%, Eq. (1) gives 10.0 ms per token and Eq. (2) 13.0. The deployed cache takes 20.6 ms, MIN-1R 15.2, and the read-ahead oracle 13.8.

Read each admitted expert once (registered: gain bands per budget on two of the machines). MIN-1R is faster than the deployed cache at every budget on the three factorial machines (Fig. 3). The usual ways of spending foresight waste reads. Prefetching without bypass, Belady’s original rule, admits experts MIN would bypass. Serving a miss on the CPU before copying it reads every admission twice. At the smallest budget of each model these gain at most 15% or lose, a trade-off known for disks (Cao et al., 1995) and CPU caches (Jain & Lin, 2018).

Admit as rarely as optimality allows (registered: faster than the deployed cache). The greedy schedule admits whenever MIN allows and may evict an expert again before its next use. The fewest-admission schedule makes 0.60–0.73 of the greedy one’s copies for the same hits in replay. Read once in the step at gpt-oss 11% (Few-1R), it beat the deployed cache on all 13 machines that ran it stably, by a geometric mean of 1.24× (registered on 7 of them). Counting the machines that ran unsteadily as well, it lost on two, the two with the slowest links (Appendix I).

6 WITHOUT FORESIGHT, AND ON A SECOND CARD

The oracles show what foresight is worth on the machines that measured it. Two questions follow: how much of it a policy without foresight can reach, and whether the machine’s link-to-CPU ratio predicts the oracles’ speed on hardware the trend was not fitted on. Both were registered before their machines were rented (Table 6).

6.1 The engine’s own mechanisms recover little

Two of the engine’s mechanisms act on the same levers without foresight. Margin, a larger admission margin, admits less: it cuts the deployed cache’s host reads by 4–6% at 11%. LA, the layer-ahead copy, applies layer $l+1$ ’s router to layer l ’s input and copies the top predicted expert ahead, so that an admission is read once, by the GPU. LA-1R adds fetching every admission in the step, and LA-1R-margin also admits less. We registered that none of the four would run faster than 1.10× the deployed cache or capture more than 35% of the read-ahead oracle’s gain on any machine. At 11% the best of them runs 1.007–1.024× the deployed cache, at most 6% of the read-ahead oracle’s gain. The counters say why: they still read 1.66–1.68 R^* host experts per token, as many as the deployed cache. A prediction one layer ahead moves reads earlier but does not avoid them. On the slow-link RTX 4090s of Section 6.2, the variants built on the layer-ahead copy lose time (0.64–0.72× at 11%), as registered, and Margin is the best, at 1.033–1.041×: up to 26% of an oracle gain that is smaller there.

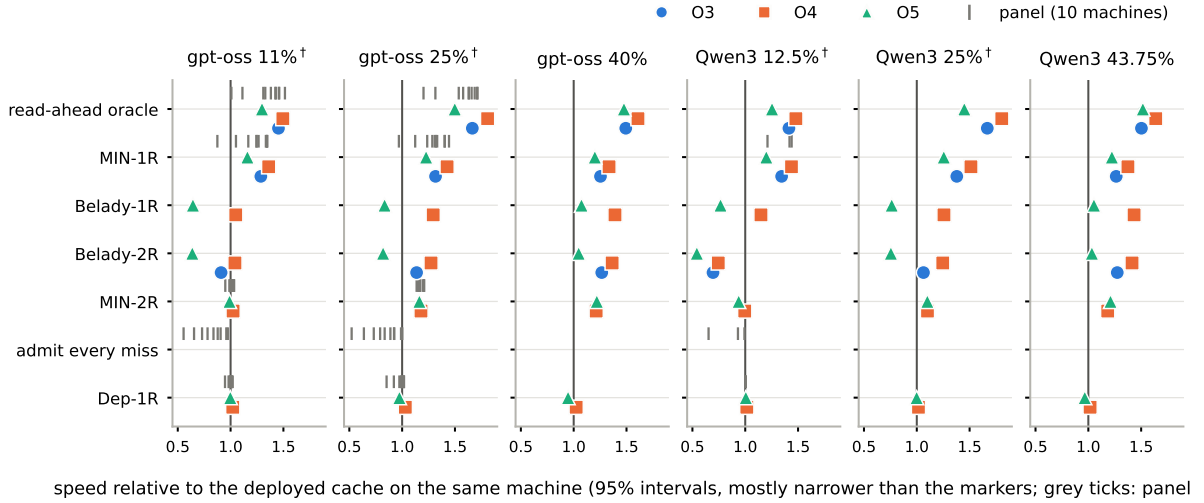


Figure 3. What foresight is worth, by how it is spent: speed of each configuration of Table 2 relative to the deployed cache on the same machine, at the six budgets ([†]host-bound). Three machines at every budget (O3–O5: two Ryzen 9 9950X and a 9950X3D); the 10 panel machines (grey ticks) at the two host-bound gpt-oss budgets, and 3 of them at Qwen3 12.5%. MIN-1R gains at every budget on these machines; reading twice or admitting what MIN bypasses gains at some budgets and loses at others.

6.2 The link-to-CPU ratio on a second card

On the RTX 5090 machines, the gain of every configuration that moves reads from the CPU to the link follows the link-to-CPU ratio: for MIN-1R, Spearman 0.89 [0.63, 0.97] over 19 machines (Fig. 4). We froze that trend on the 15 panel machines, one line in the log of the ratio for each of MIN-1R, MIN-2R, Dep-1R and the read-ahead oracle, and registered that their speed on RTX 4090 machines, a slower card behind a PCIe 4.0 link, would fall within 0.10 of it in log. The first test ran on three machines, all at ratios 0.37–0.40; a second, with the same predictions, added two at 0.57–0.77. On all five, MIN-1R and the read-ahead oracle came within 7% of the prediction at 11% and 5% at 25% (Table 6), and every machine’s 95% interval over problems lies inside the band. On the two with fast links the interaction of Section 4 is positive, as registered.¹ Simpler predictors do worse on the same machines: no change ($1.0\times$) misses by up to 46% at 11% and 69% at 25%, and the mean of the two slow-link panel machines by up to 38% and 34%. Applied after the fact to the new RTX 5090s of Section 4, all fast-linked, the same trend misses five of their 18 cells by more than the band, by up to 18%: on fast links the ratio sets the direction of the gain, not its size. At the slow-link RTX 4090s’ ratios MIN-1R runs $0.96\text{--}1.01\times$ the deployed cache

¹The first launch of these machines stopped after one round: its output check used the RTX 5090’s reference loss, and the RTX 4090 computes a loss 3% higher in every configuration. We relaunched them with the card’s own reference and unchanged predictions; the first rounds, in the repository by then, had come within 6% of the trend (Appendix D).

at 11%, and $1.13\text{--}1.31\times$ on the two at higher ratios; on the RTX 5090s whose link reads at under a third of their CPU rate (ratios 0.28–0.32), Few-2R paid instead ($1.11\text{--}1.22\times$ over the two budgets).

7 HOW MUCH FORESIGHT IS NEEDED

The oracles know the whole future; a predictor would know part of it, imperfectly. We give the cache a *window*, the routing of the next W tokens, exact or with each future expert right only with probability r . Throughout this section, the value of a window is its share of MIN’s gain over admitting every miss, the online policy that reads least at these budgets (Appendix G gives the policies).

Next-token prediction buys little. Exact routing of the next token recovers at most 0.31 of MIN’s gain at any host-bound budget of our two models; half of the gain at gpt-oss 11% needs 4 tokens (Fig. 5). In the engine, an exact 16-token window recovers 0.78–0.93 of the gain in time at 11% (registered at 0.80 or more; the lowest machine missed). Against the deployed cache, the yardstick of Section 5, the same window copied in the step runs $0.89\text{--}1.32\times$ at 11% on 4 machines, below 1 on the 2 with the lowest link-to-CPU ratios (up to 0.54): admitting every miss is a weak base where the link is relatively slow.

Accuracy matters as much as horizon. An 8-token window that is right half the time recovers 0.29 of the gain at 11%, about what 2 exact tokens buy (0.31).

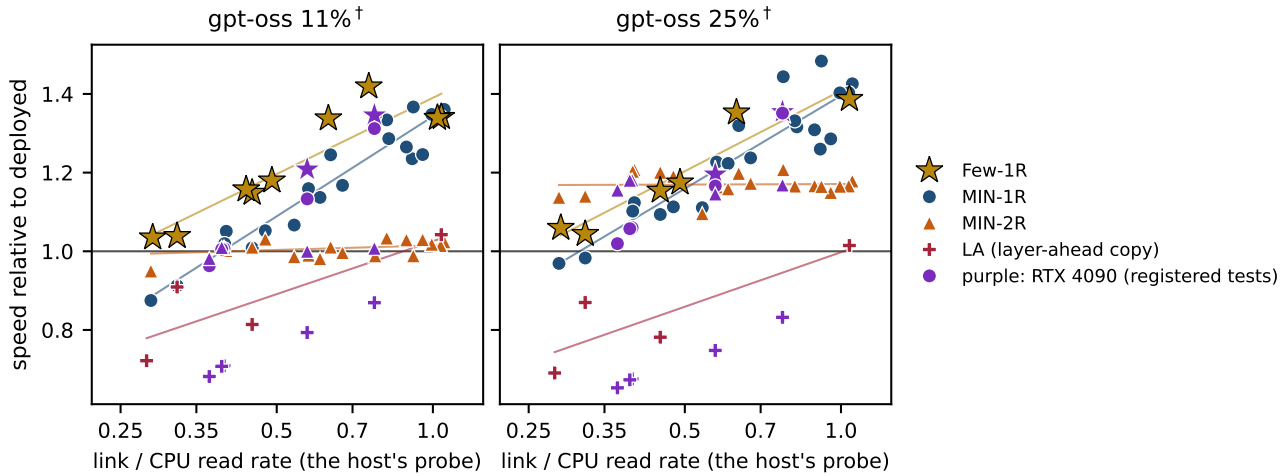


Figure 4. The machine decides which way of spending foresight pays: speed relative to the deployed cache on the same machine against the machine’s link-to-CPU ratio, at the two host-bound gpt-oss budgets. Every marker is a machine that passed our checks. Stars: Few-1R. Circles and triangles: MIN-1R and MIN-2R, both on the greedy schedule. Crosses: LA. Purple markers: the RTX 4090 machines of the registered second-card test (Section 6.2), not used in the fits. Lines: least-squares fits in the log of the ratio, drawn for the trend only.

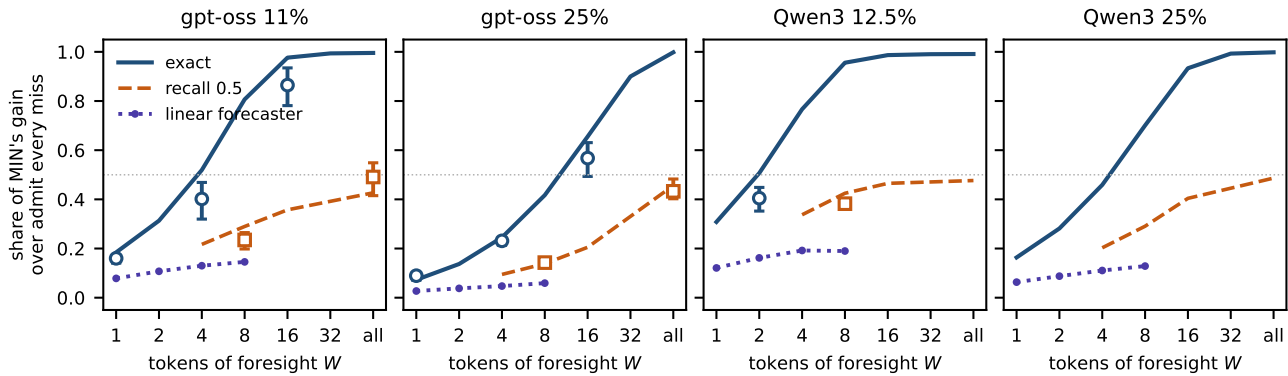


Figure 5. Baseline here: *admit every miss*, not the deployed cache. The value of partial foresight at the four host-bound budgets: the share of MIN’s gain over admitting every miss recovered by exact routing of the next W tokens (solid), by a window whose experts are each right with probability 0.5 (dashed), and by a linear forecaster fitted on other text (dotted), in host reads on the routing the engine runs. Markers: the same share in time per token on the panel machines (mean and range over machines; 10 machines for gpt-oss, 3 for Qwen3 at 12.5%). The grey line marks half of the gain.

The horizon is measured in distinct experts (exploratory). On 9 models’ own sampled text, the window that closes half of the read gap ranges from 0.6 to 34 tokens. Paging theory measures lookahead in distinct pages (Albers, 1997; Breslauer, 1995), and so does the rule that fits: within that window a layer requests about $0.65C$ distinct experts (0.66 – $0.81C$ on the AIME routing the engine runs). Given a model’s routing trace, this one-constant rule predicts its horizon as well as a two-constant power law in tokens (Appendix F).

Nothing realisable reaches it. Forecasters trained on routing histories, linear or recurrent, close at most 15% of the read gap on gpt-oss: their precision falls with the horizon. Speculative batches save an online cache about a tenth of its reads at 90% draft acceptance and cost reads at 80% (Chen et al., 2025b; Li et al., 2026; Saxena et al., 2026) (Appendix F). Layer-ahead predictors (Hwang et al., 2024; Zhu et al., 2025) reach 84% recall in our engine but see less than one token ahead; built into a policy without foresight, they recover at most 6% of the oracle’s gain (Section 6.1).

8 WHAT THIS MEANS FOR SYSTEM BUILDERS

1. **Report the distance to the machine’s bound.** A speed-up over a chosen baseline hides how much is left. Equation (1) needs only the routing trace and one probe run; on our consumer machines the best system reaches 31–54% of its speed at the smallest gpt-oss budget.
2. **Change what is cached and how it is read together.** Serving a miss on the CPU and then copying it into a slot reads it twice. A better set read twice, or the same set read once, closes almost none of the gap; where the link reads at least half as fast as the CPU, a better set read once closes about a third of it.
3. **Do not expect a policy without foresight to get there.** Admitting less and predicting one layer ahead recover at most 6% of what foresight is worth where it is worth most, and the layer-ahead copy loses time on slow links; the reads they leave are the ones only foresight avoids.
4. **Choose the read path by the link-to-CPU ratio.** Where the link is fast, copy misses into slots in the step; where it is slow, serve them on the CPU and copy in the background. On a second card the ratio predicted the oracles’ speed within 7% over ratios 0.37–0.77. The probe that measures the ratio takes seconds.
5. **Issue reads before routing, and predict far enough ahead to save reads.** Even with full foresight, the GPU’s non-expert time stays in series with every read that waits for the router. Only a prediction of the routing lets reads start earlier, and only over the link (Eq. (3)). To save reads, predictors should target the next $\approx 0.65 C$ distinct experts of each layer, not the next token; accuracy counts as much as horizon.

9 RELATED WORK

Offloading systems. Demand-paging systems cache experts per layer and copy misses over PCIe (Eliseev & Mazur, 2023; Xue et al., 2024; Song et al., 2024a; Tang et al., 2024; Kong et al., 2024; Zhong et al., 2024; Yi et al., 2023; He et al., 2024; Shams, 2026), or use predicted expert activations to decide what to keep (Du et al., 2024); an upstream llama.cpp pull request adds such a cache, admitting every miss into an LRU (Gupta, 2026). CPU-execution systems run non-resident experts on the host (Kamahori et al., 2024; Chen et al., 2025a; Zhong et al., 2025; Zhang et al., 2025; Wang et al., 2026a; ATSIInfer), as PowerInfer runs a dense model’s cold neurons (Song et al., 2024b); a set-associative GPU cache with CPU-executed misses (Huang et al., 2025), a llama.cpp fork (leloch, 2026a;b) and two llama.cpp pull requests (miltos22, 2026; csantiago78, 2026) are the closest

designs to ours. FreeToken (Yang et al., 2026) and DALI (Zhu et al., 2026) split each layer’s misses between CPU and PCIe. Predictors guess upcoming experts from the previous layer’s state (Hwang et al., 2024; Zhu et al., 2025), several tokens ahead with a learned forecaster (Wang et al., 2026b; Yadav & Asgari, 2026), or from draft tokens (Chen et al., 2025b; Li et al., 2026; Saxena et al., 2026); Read-ME decouples the router so routing is known in advance (Cai et al., 2024). Batched-throughput systems (Sheng et al., 2023; Cao et al., 2024; Yuan et al., 2025) and flash or SSD tiers (Alizadeh et al., 2024; Liang et al., 2026a; Lin et al., 2026) are outside our scope.

Bounds, oracles and caching theory. WiSP (Zhang et al., 2026b) bounds decode by PCIe turnover, Budgeting Bytes (Zhang, 2026a) by bytes over bandwidth for a given fetch set, and MoE-CAP (Jiang et al., 2025) measures bandwidth utilisation against a single peak; the roofline (Williams et al., 2009) is the single-memory ancestor of our bound. Zhang (2026b) propose a trace-level evaluation contract and report a 44–46% gap between causal policies and MIN with bypass; Liang et al. (2026b) measure, across 20 models, the hit rate of an oracle cache that sees m tokens ahead. We add the time domain: oracles inside a real engine, a measured decomposition of the gap, the cost of how foresight is spent, and a horizon rule in distinct experts. SAEM (Zhang et al., 2026a) gives its own stage-level planner perfect knowledge of the coming stage and finds that worth at most about 14% on an A100; SpecMD (Hoang et al., 2026) benchmarks expert-cache and speculative prefetch policies across hardware; Angelopoulos et al. (2025) give competitive bounds for layered expert paging. With bypass, MIN maximises hits exactly (Carlisle & Lloyd, 1995; Jain & Lin, 2016); when bypass and admission cost differ, the time-optimal placement is a different problem (Zhang et al., 2020), which is why we use MIN’s reads as an input to a bound rather than as a time-optimal policy. Choosing among hit-optimal schedules by what admissions cost has precedents: Demand-MIN for prefetching (Jain & Lin, 2018), offline-optimal caching as a min-cost flow over reuse intervals (Berger et al., 2018), and admission rationing in flash caches (Eisenman et al., 2019). Our fewest-admission schedule is that interval formulation with admissions as a second objective, measured in an engine. Learning-augmented paging is most robust with predictions of whether a page will be evicted before its next use (Antoniadis et al., 2023); learned replacement imitates Belady in CPU and content-delivery caches (Jain & Lin, 2016; Song et al., 2020) and in an SSD-backed expert cache (Kim et al., 2026).

10 LIMITATIONS

- **Hardware and sample.** One GPU model for most results, rented, mostly in desktop hosts, and a second

(RTX 4090) for two registered tests; clocks could not be locked and power limits differed. The first RTX 4090 test deviated from its registration in three ways: its output check used the RTX 5090’s reference loss, so its machines were relaunched with the RTX 4090’s; two relaunches that failed to download the model were repeated once; and one of the four machines eligible for relaunch was not relaunched. In the last experiment one machine never finished downloading the model, and its replacement started before it was given up (Appendix D). The machines are a convenience sample of the cheapest offers that passed our checks, and few have a link slower than half their CPU rate, so where the crossovers lie rests on few points. Rented machines are often shared or unsteady: in the two tests of the time relation on new machines, 3 and 2 machines stopped at the gate and 2 and 1 ran unsteadily, and of 8 launches on server processors only three passed every check.

- **Workload.** Batch-1 decode of two models on AIME prompts, also used while developing the system, teacher-forced on each model’s own greedy text. Prompts and batches above one bypass the cache.
- **The bounds.** Exact routing and per-layer slots; pooling slots across layers lowers MIN’s reads by 4.5–18.6%. MIN’s count lets an expert served in a step be evicted later in that step; keeping it for the step raises the bound slightly (Table 28). R^* is counted on the 30-problem trace; on the first 20 problems, which the later experiments time, it changes by -0.4% . The bounds drop latencies and are relative to one run of our probe, which on most machines ran while the model was downloading in the background: by the closed-form account of Appendix H, the engine’s reads imply a rate more than 5% above the probe’s highest reading on 12 of 84 launch-budgets (Table 27). In the last experiment a second probe, run on the idle machine after the timed runs, came within 4.9% of the first on all 4 machines; between two rentals of the same machine the probed link-to-CPU ratio moved by up to 12%. T_{GPU} is our engine’s kernel time, not a property of the machine alone.
- **The decomposition.** The parts are measured at two budgets of one model. What the oracles leave is split by attribution, not measurement: the GPU’s non-expert time is profiled, not removed. The machines pool runs of 20 and 30 problems and three versions of the engine. Equation (3) assumes that the CPU reads an expert’s weights when it runs the expert.
- **The oracles.** Oracles read a recorded trace. In the engine the fewest-admission schedule misses 2.5–5.3% more than the greedy one, more than its replay predicts; its measured gains include that cost. Degraded

windows draw errors independently, while a real predictor’s errors correlate with what the cache holds. The only predictor we built into the engine looks one layer ahead; the multi-token forecasters of Section 7 were evaluated offline, on host reads.

- **Outputs.** Parity is measured by teacher-forced KL divergence against an all-VRAM reference (mean 0.0019 and 0.0005 nats), not by task accuracy.

11 CONCLUSION

MIN’s host reads at a machine’s probed rate bound batch-1 MoE decode with experts in host memory, and the bound’s read time is nearly reachable. Our cache runs at 31–54% of it at the smallest gpt-oss budget on machines with consumer processors, and the other systems we measured run further from it. Oracles inside the engine show where the rest goes: caching MIN’s set and reading each admitted expert once pay only together, a result we registered and replicated on machines rented for the test. Policies without foresight built from the engine’s own mechanisms recover little of it, because they read as much as the deployed cache. Which read path pays is set by the machine’s link-to-CPU ratio, and a trend in that ratio predicted the oracles’ speed on a second card. What the oracles leave we attribute to the GPU’s own work, in series with reads that wait for routing, and to the oracle’s extra reads. Foresight’s value has a price: half of it needs the routing of about 0.65 C distinct experts per layer ahead, 4 tokens on our workload at the smallest budget.

REFERENCES

- Albers, S. On the influence of lookahead in competitive paging algorithms. *Algorithmica*, 18(3):283–305, 1997. doi: 10.1007/PL00009158.
- Alizadeh, K., Mirzadeh, I., Belenko, D., Khatamifard, K., Cho, M., Del Mundo, C. C., Rastegari, M., and Farajtabar, M. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Angelopoulos, S., Marchal, L., Obrecht, A., and Simon, B. Cache management for mixture-of-experts LLMs. In *Euro-Par*, 2025. arXiv:2509.02408.
- Antoniadis, A., Boyar, J., Eliáš, M., Favrholt, L. M., Hoeksma, R., Larsen, K. S., Polak, A., and Simon, B. Paging with succinct predictions. In *International Conference on Machine Learning (ICML)*, 2023.
- ATSIInfer. ATSIInfer: Automated tensor scheduling for hybrid CPU-GPU LLM inference on consumer devices, 2026. arXiv:2607.10183.

- Belady, L. A. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2): 78–101, 1966.
- Berger, D. S., Beckmann, N., and Harchol-Balter, M. Practical bounds on optimal caching with variable object sizes. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (SIGMETRICS)*, 2(2), 2018.
- Breslauer, D. On competitive on-line paging with lookahead. Technical report, BRICS, 1995.
- Cai, R., Ro, Y., Kim, G.-W., Wang, P., Ehteshami Bejnordi, B., Akella, A., and Wang, Z. Read-ME: Refactorizing LLMs as router-decoupled mixture of experts with system co-design. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pp. 116126–116148, 2024. arXiv:2410.19123.
- Cao, P., Felten, E. W., Karlin, A. R., and Li, K. A study of integrated prefetching and caching strategies. In *Proceedings of ACM SIGMETRICS*, 1995.
- Cao, S., Liu, S., et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- Carlisle, M. C. and Lloyd, E. L. On the k -coloring of intervals. *Discrete Applied Mathematics*, 59, 1995.
- Chen, H., Xie, W., Zhang, B., Tang, J., Wang, J., Dong, J., Chen, S., Yuan, Z., Lin, C., Qiu, C., Zhu, Y., Ou, Q., Liao, J., Chen, X., Ai, Z., Wu, Y., and Zhang, M. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025a. doi: 10.1145/3731569.3764843.
- Chen, L., Wen, Z., Wu, T., Zhang, X., and Wu, C. SP-MoE: Speculative decoding and prefetching for accelerating MoE-based model inference. *arXiv preprint arXiv:2510.10302*, 2025b.
- csantiago78. llama.cpp pull request #27861: GPU-resident LRU cache for host-offloaded MoE expert weights. <https://github.com/ggml-org/llama.cpp/pull/27861>, 2026. Draft, accessed September 2026.
- Du, Z., Li, S., Wu, Y., Jiang, X., Sun, J., Zheng, Q., Wu, Y., Li, A., Li, H., and Chen, Y. SiDA: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- Eisenman, A., Cidon, A., Pergament, E., Haimovich, O., Stutsman, R., Alizadeh, M., and Katti, S. Flashield: a hybrid key-value cache that controls flash write amplification. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019.
- Eliseev, A. and Mazur, D. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- Fang, Z., Hong, Z., Huang, Y., Lyu, Y., Chen, W., Yu, Y., Yu, F., and Zheng, Z. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. *arXiv preprint arXiv:2502.12224*, 2025.
- Gerganov, G. and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- Gupta, A. llama : add a GPU cache for MoE experts kept in host memory. llama.cpp pull request 29887, <https://github.com/ggml-org/llama.cpp/pull/29887>, 2026. Opened 3 October 2026.
- He, X., Zhang, S., Tang, K., Shi, S., Wang, Y., Zeng, Z., Tang, Z., Chu, X., Yin, H., Tsang, I. W., and Ong, Y. S. ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling. *arXiv preprint arXiv:2410.17954*, 2024. Accepted at DAC 2026 (per arXiv v2).
- Hoang, D., Jaiswal, A., Samragh, M., and Cho, M. SpecMD: A comprehensive study on speculative expert prefetching. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03921.
- Hoefer, T. and Belli, R. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- Hoefer, T., Alistarh, D., Ben-Nun, T., Dryden, N., and Peste, A. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- Huang, E.-M., Lin, L.-S., et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- Hwang, R., Wei, J., et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. arXiv:2308.12066.
- Ivanov, A., Dryden, N., Ben-Nun, T., Li, S., and Hoefer, T. Data movement is all you need: A case study on

- optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- Jain, A. and Lin, C. Back to the future: Leveraging Belady’s algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016.
- Jain, A. and Lin, C. Rethinking Belady’s algorithm to accommodate prefetching. In *International Symposium on Computer Architecture (ISCA)*, 2018.
- Jiang, Y., Fu, Y., Huang, Y., et al. MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2412.07067.
- Kalibera, T. and Jones, R. Rigorous benchmarking in reasonable time. In *International Symposium on Memory Management (ISMM)*, 2013.
- Kamahori, K., Tang, T., et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- Kim, B., Lee, J., Han, D., Yoo, H.-J., and Kim, S. Flash-MoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices. *arXiv preprint arXiv:2601.17063*, 2026.
- Kong, R., Li, Y., Feng, Q., Wang, W., Ye, X., Ouyang, Y., Kong, L., and Liu, Y. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- leloch. MoE expert cache for llama.cpp: VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. <https://github.com/leloch/llama.cpp/tree/moe-cache-v2-pr>, 2026a. Branch moe-cache-v2-pr, commit e3096b0 (6 August 2026). Proposed upstream as llama.cpp pull request #24524 and RFC discussion #24528; not merged. Accessed September 2026.
- leloch. RFC: MoE expert cache, VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. llama.cpp discussion #24528, <https://github.com/ggml-org/llama.cpp/discussions/24528>, 2026b. Posted 12 June 2026; accessed September 2026.
- Li, S., Lin, J., Ge, D., and Ye, Y. MoE-SpAc: Efficient MoE inference based on speculative activation utility in heterogeneous edge scenarios. *arXiv preprint arXiv:2603.09983*, 2026.
- Liang, F., Shen, Y., Hu, J., Xu, J., Gao, H., Xu, M., and Meng, Z. SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC. *arXiv preprint arXiv:2609.18110*, 2026a.
- Liang, J. et al. Not all models suit expert offloading: On local routing consistency of mixture-of-expert models. In *International Conference on Learning Representations (ICLR)*, 2026b. arXiv:2505.16056.
- Lin, Y., Wang, Y., Cai, R., Liu, H., and Zeng, X. The other half of the memory wall: Serving 35B MoEs from SSD with trained routing prediction. *arXiv preprint arXiv:2609.18063*, 2026.
- miltos22. llama.cpp pull request #26563: Expert caching that greatly increases performance (self-contained, off by default). <https://github.com/ggml-org/llama.cpp/pull/26563>, 2026. Accessed September 2026.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- Saxena, A., Tsai, P.-A., Taneja, H., Jaleel, A., and Qureshi, M. Cascade: Utility-driven speculative decoding for mixture-of-experts. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026. arXiv:2506.20675 (2025).
- Shams, M. Paging the experts: A reproducible characterization of flash-backed MoE inference on iPhone. *arXiv preprint arXiv:2609.29032*, 2026.
- Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., and Zhang, C. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pp. 31094–31116, 2023. arXiv:2303.06865.
- Song, X., Zhong, Z., et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024a.
- Song, Y., Mi, Z., Xie, H., and Chen, H. PowerInfer: Fast large language model serving with a consumer-grade GPU. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2024b.
- Song, Z., Berger, D. S., Li, K., and Lloyd, W. Learning relaxed Belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- Tang, P., Liu, J., et al. HOBbit: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.

- Ukarande, A., Shekhar, D., Blackstein, M., and Rangan, R. Efficient, VRAM-constrained xLM inference on clients. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 8, 2026. Industry track. arXiv:2604.26334. Artifact: <https://github.com/deepshnv/pipeshard-mlsys26-ae>, tag v2.0.3-mlsys26.
- Wang, W., Liu, J., Hou, X., Xia, X., Tang, P., Zhang, M., Li, C., and Guo, M. MoE-SpeQ: Speculative quantized decoding with proactive expert prefetching and offloading for mixture-of-experts. *arXiv preprint arXiv:2511.14102*, 2025.
- Wang, W., Hou, Y., et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026a.
- Wang, Z., Wang, Y., et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026b.
- Williams, S., Waterman, A., and Patterson, D. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- Xue, L., Fu, Y., et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- Yadav, S. and Asgari, B. Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging. *arXiv preprint arXiv:2609.38090*, 2026.
- Yang, A. et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, S., Fan, X., Pan, M., Xi, H., Wang, Z., Sun, S., Keutzer, K., Han, S., Zaharia, M., Xu, C., and Stolica, I. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.
- Yi, R., Guo, L., Wei, S., Zhou, A., Wang, S., and Xu, M. EdgeMoE: Empowering sparse large language models on mobile devices. *arXiv preprint arXiv:2308.14352*, 2023.
- Yuan, Y., Ma, L., et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- Zhang, Gao, and Mitra. SAEM: Stage-aware expert management for memory-efficient MoE inference in chain-of-thought reasoning, 2026a. arXiv:2608.21614.
- Zhang, H. Budgeting bytes: A windowed storage roofline and dual-budget architecture ablations for storage-bound LLM decoding. *arXiv preprint arXiv:2609.04238*, 2026a.
- Zhang, J., Wu, L., Darbari, M., and Hong, L. WiSP: A working-set view of mixture-of-experts serving on extremely low-resource hardware. *arXiv preprint arXiv:2606.21868*, 2026b.
- Zhang, L., Karimi, R., Ahmad, I., and Vigfusson, Y. Optimal data placement for heterogeneous cache, memory, and storage systems. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, 4(1):6:1–6:27, 2020. doi: 10.1145/3379472.
- Zhang, Y. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026b.
- Zhang, Y., Aggarwal, S., and Mitra, T. DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference. In *Design, Automation & Test in Europe Conference (DATE)*, pp. 1–7, 2025. doi: 10.23919/DATE64628.2025.10992741. arXiv:2501.10375.
- Zhong, S., Liang, L., Wang, Y., Wang, R., Huang, R., and Li, M. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.
- Zhong, S. et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. arXiv:2504.05897.
- Zhu, S., Bohl, S., et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.
- Zhu, Z., Li, G., Wang, P., Mo, Z., Pei, M., Song, Z., Liang, X., and Cheng, J. DALI: A workload-aware offloading framework for efficient MoE inference on local PCs. *arXiv preprint arXiv:2602.03495*, 2026.

Table 1. Our claims, how each was tested, and on how many machines. *Pre-specified*: predicted, with its threshold, in a job script committed before the machines started (Appendix D scores every one); *post hoc*: found in the data it describes; *derived*: follows from the definitions. *Mach.*: machines that passed every check; “15 + 5” is the panel and, separately, the machines rented later for the registered test.

Claim	Test and outcome	Mach.	§
MIN’s reads at the probed rate bound any exact-routing cache with C slots per layer	derived	–	3
The bound’s read time is nearly reachable	pre-specified: $\geq 50\%$ of it per layer; 5 of 33 clauses failed	3	3
Our cache is faster than FreeToken and llama.cpp at most configurations	pre-specified ratios on host B, held; repeated on host S they fell outside ± 0.06 of B’s at five of six configurations, still faster at 11 of 12 in all	B, S	3
MIN’s set and one read pay only together (gpt-oss 11%, fast links)	pre-specified: interaction > 0 on every machine; held on 9 of 10 panel machines and on 5 of 5 new ones	15 + 5	4
Landing the copies in time does not rescue the two-read path	pre-specified: misses $\leq 0.95\times$ and no slowdown, failed and failed; second read costs more than the landing, held	4	4
Policies without foresight recover little of the oracles’ gain	pre-specified: capture $\leq 35\%$ per machine; at most 6%	5	6.1
The link-to-CPU ratio predicts the oracles’ speed on a second card	trend fitted post hoc on RTX 5090s; pre-specified on RTX 4090s at ratios 0.37–0.77; within 0.10 in log, held (7%); misses new fast-link RTX 5090s by up to 18%	5	6.2
MIN’s fewest-admission schedule gains on every machine that ran it stably	pre-specified on 7 machines: > 1 , held	13	5
Published systems reach a median 13.6% of their bound	post hoc; their papers’ hardware at datasheet rates	–	3
Half of foresight’s value needs $\approx 0.65 C$ distinct experts ahead	post hoc, on 9 models’ traces	–	7
A closed-form account of the time	post hoc; three of four pre-specified tests failed	–	H

Table 2. Cache configurations. *Reads*: how many times an admitted expert is read from host memory (1R: fetched into its slot in the step; 2R: served by the CPU, then copied in the background). All run in the same engine; the oracles read the future routing from a trace of the same text recorded on the same machine.

Name	What it admits, and how	Reads
<i>No foresight</i>		
deployed cache	decayed request count beats the lowest resident’s by $\kappa=1$; the CPU serves the miss, then a background copy fills the slot	2
Dep-1R	the same admissions, fetched into the slot in the step	1
admit every miss	every miss fetched into the lowest-scored slot (demand fetch)	1
Margin	the deployed cache with a larger margin ($\kappa=3$ at 11%, 2 at 25%); it admits less the deployed cache plus a copy, one layer ahead, of the non-resident expert that the next layer’s router, applied to this layer’s input, ranks highest	2
LA	the deployed cache plus a copy, one layer ahead, of the non-resident expert that the next layer’s router, applied to this layer’s input, ranks highest	2 or 1
LA-1R, LA-1R-margin	Dep-1R plus the layer-ahead copy; the second also with Margin’s margin	1
<i>Oracles</i>		
MIN-2R	MIN with bypass (greedy rule); the CPU serves the miss, then a copy	2
MIN-1R	MIN with bypass (greedy rule); admissions fetched in the step	1
Few-1R, Few-2R	the hit-optimal schedule with the fewest admissions (an LP per machine), read once or twice	1, 2
Few-2R-early	Few-2R with each copy issued one step earlier, so that it lands for the next use	2
read-ahead oracle	MIN’s admissions copied three steps ahead where the next use allows, else fetched in the step; the early copies evict sooner, so it reads 1.22–1.31 R^* at 11%	1
Belady-2R, Belady-1R	every expert of the coming steps, into the slot needed furthest ahead: Belady’s rule without bypass (the second also read ahead)	2, 1
window W	admit every miss unless the next W tokens show the victim needed sooner; victim the lowest-scored resident not seen in the window, else the one seen furthest	1
window W , recall r	the same, each future expert kept with probability r , else replaced by a random one	1
deployed + window W	the deployed cache with the same window and victim choice; the CPU serves the miss, then a background copy	2

Table 3. Decode speed at equal GPU expert memory on two RTX 5090 hosts (30 AIME-25 problems, the first 256 greedy tokens, one session). Ratios are of mean speeds; the ratio over FreeToken is paired by problem within one launch, with a 95% bootstrap interval over problems. Host B’s rows come from three rentals of the same machine (the same GPU), so its ratios over llama.cpp compare launches. Each system decodes its own greedy text. Our cache uses each machine’s fetch table; FreeToken uses its faster backend per budget, picked on host B on a separate launch. *Bound*: Eq. (1) on that machine (MIN’s reads, the host’s highest probed rate) with the GPU at its datasheet rate, and at the rate all-in-VRAM decode measures at batch size 1 (52% of datasheet for gpt-oss, 61% for Qwen3; Table 28 varies the other assumptions). Against the pooled bound their designs allow, FreeToken (one cache for all layers) and llama.cpp (whole layers pinned) stand at 18–40% and 8–20%. With every weight in GPU memory (an RTX PRO 6000) stock llama.cpp decodes gpt-oss at 255 and Qwen3 at 178 tok/s. [†]Selection launch only, which favours FreeToken.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Bound, datasheet tok/s	GPU ours	Bound, measured tok/s	GPU ours
Host B: Ryzen 9 9950X3D, card at 17001 MHz memory clock, CPU 72 GB/s, link 53, together 78, highest (the bound’s rate) 88										
gpt-oss-120b	11%	34.9	54.0 (hybrid)	69.9	1.294 [1.278, 1.312]	2.00×	172	41%	172	41%
gpt-oss-120b	25%	40.0	85.6 (hybrid)	109.2	1.275 [1.254, 1.295]	2.73×	430	25%	279	39%
gpt-oss-120b	40%	47.7	132.2 (offload)	152.5	1.154 [1.134, 1.172]	3.20×	519	29%	279	55%
Qwen3-30B-A3B	12.5%	19.5	38.7 (hybrid)	40.0	1.032 [1.022, 1.042]	2.05×	96	42%	96	42%
Qwen3-30B-A3B	25%	22.1	54.7 (hybrid)	63.1	1.153 [1.134, 1.173] [†]	2.86×	214	30%	192	33%
Qwen3-30B-A3B	43.75%	28.4	103.1 (offload)	108.2	1.049 [1.036, 1.064]	3.81×	308	35%	192	56%
Host S: Ryzen 9 9950X at the stock clock, card at 14001 MHz, CPU 56 GB/s, link 53, together 66, highest 71; FreeToken’s backend carried over from host B										
gpt-oss-120b	11%	28.9	48.0 (hybrid)	57.9	1.207 [1.195, 1.219]	2.00×	140	41%	140	41%
gpt-oss-120b	25%	33.6	78.7 (hybrid)	94.1	1.196 [1.163, 1.224]	2.80×	351	27%	275	34%
gpt-oss-120b	40%	40.1	122.2 (offload)	133.6	1.093 [1.077, 1.111]	3.34×	514	26%	275	49%
Qwen3-30B-A3B	12.5%	16.1	32.7 (hybrid)	33.5	1.027 [1.018, 1.036]	2.08×	78	43%	78	43%
Qwen3-30B-A3B	25%	18.5	51.4 (hybrid)	53.9	1.048 [1.037, 1.059]	2.91×	174	31%	174	31%
Qwen3-30B-A3B	43.75%	23.8	98.3 (offload)	95.8	0.974 [0.962, 0.987]	4.02×	305	31%	189	51%

Table 4. The deployed cache’s gap to Eq. (1), split by oracles in the engine (Fig. 2): mean share of the gap, in percent, with 95% t -intervals over machines. *Panel*: the 15 machines of jobs 096–101 that ran every state. *New*: the 5 machines of job 109, rented for this test, whose population (desktop-class, link-to-CPU ratio at least 0.5) and predictions were registered before any of them started; the job’s deadline cut one machine’s 25% round. The first two rows each change one thing; the third changes both. *Both, then the read-ahead oracle and left* sum to the gap. The last three rows are attributed, not measured: they split *left* with T_{GPU} as in Eq. (2) (2.9 ms, the smallest profile) and the read-ahead oracle’s reads beyond R^* at B_{host} ; the residual is what they leave.

Part of the gap	gpt-oss 11%		gpt-oss 25%	
	Panel (15)	New (5)	Panel (15)	New (4)
<i>Measured</i>				
MIN’s set alone (MIN-2R)	0 [−2, 3]	3 [−1, 8]	21 [19, 22]	20 [18, 21]
One read alone (Dep-1R)	0 [−2, 2]	0 [−2, 1]	−3 [−7, 1]	−2 [−3, −1]
Both (MIN-1R)	33 [21, 44]	35 [22, 48]	31 [23, 39]	27 [23, 32]
Then the read-ahead oracle	15 [11, 19]	19 [14, 24]	21 [18, 23]	24 [21, 27]
Left after all three	52 [43, 62]	46 [38, 55]	48 [41, 55]	48 [44, 53]
<i>Attributed: of what is left</i>				
the GPU’s non-expert time (T_{GPU})	27 [25, 30]	28 [20, 36]	34 [31, 37]	38 [33, 43]
the oracle’s reads beyond MIN’s, at B_{host}	21 [20, 23]	18 [15, 20]	19 [18, 21]	16 [15, 17]
residual	4 [−6, 13]	1 [−3, 4]	−5 [−12, 2]	−5 [−7, −3]

Table 5. When the copy lands, against how often the expert is read: MIN’s fewest-admission set read once (**Few-1R**), read twice with the copy published two steps after the admission (**Few-2R**), and read twice with each copy issued a step earlier so that it lands for the next use (**Few-2R-early**). Speed relative to the deployed cache on the same machine (every 95% interval over problems within ± 0.01) and host misses per token, at gpt-oss 11%; registered before the machines were rented.

Machine	Ratio	Speed vs deployed			Misses per token		
		1R	2R	early	1R	2R	early
<i>RTX 5090</i>							
Ryzen 9 5950X	0.74	1.41	1.03	0.98	40.7	55.6	54.5
Ryzen 9 7945HX	0.88	1.25	1.03	1.00	40.7	55.7	54.6
<i>RTX 4090</i>							
Ryzen 7 7800X3D	0.57	1.21	1.03	1.00	40.9	54.7	53.0
Core i5-12400	0.77	1.35	1.03	0.98	40.8	55.7	54.5

Table 6. The registered tests on machines rented for them: speed relative to the deployed cache on the same machine (geometric mean of its rounds) of MIN-1R, the read-ahead oracle (*ahead*), and the best of the four variants without foresight (*none*: Margin, LA, LA-1R, LA-1R-margin; Section 6.1), and that variant’s capture, its share of the read-ahead oracle’s gain. *Ratio*: link-to-CPU. RTX 5090: job 109, whose population (desktop-class, link-to-CPU ratio at least 0.5) was fixed before any machine started. RTX 4090: job 111 (job 110’s predictions, committed before any RTX 4090 ran) and job 112 (the same predictions, on machines at higher ratios); in brackets, what the RTX 5090 machines’ trend in the link-to-CPU ratio predicted for that machine. Subscripts: half-width of the 95% interval over problems (paired bootstrap within the machine).

		gpt-oss 11%				gpt-oss 25%			
Machine	Ratio	MIN-1R	Ahead	None	Capture	MIN-1R	Ahead	None	Capture
RTX 5090, job 109									
Core Ultra 9 285K	0.54	1.22 $\pm$.01	1.46 $\pm$.01	1.02 $\pm$.00	4 $\pm$ 0%	1.24 $\pm$.01	1.64 $\pm$.02	1.03 $\pm$.00	4 $\pm$ 0%
Core i9-13900KF	0.59	1.25 $\pm$.01	1.46 $\pm$.01	1.01 $\pm$.00	2 $\pm$ 0%	1.26 $\pm$.01	1.61 $\pm$.02	1.01 $\pm$.01	2 $\pm$ 1%
Ryzen 9 5950X	0.84	1.37 $\pm$.01	1.50 $\pm$.01	1.01 $\pm$.00	2 $\pm$ 0%	—	—	—	—
Ryzen 7 9800X3D	0.88	1.15 $\pm$.01	1.34 $\pm$.01	1.01 $\pm$.01	4 $\pm$ 2%	1.22 $\pm$.01	1.55 $\pm$.02	1.01 $\pm$.00	2 $\pm$ 1%
Ryzen 9 7945HX	0.89	1.24 $\pm$.01	1.42 $\pm$.01	1.02 $\pm$.01	6 $\pm$ 1%	1.28 $\pm$.01	1.63 $\pm$.02	0.99 $\pm$.01	−1 $\pm$ 1%
RTX 4090, job 111									
Core i9-14900KF	0.37	0.96 $\pm$.00 [0.99]	1.13 $\pm$.01 [1.11]	1.03 $\pm$.00	26 $\pm$ 3%	1.02 $\pm$.00 [1.07]	1.30 $\pm$.01 [1.29]	1.04 $\pm$.00	13 $\pm$ 1%
Core Ultra 9 285K	0.39	1.01 $\pm$.00 [1.01]	1.20 $\pm$.01 [1.13]	1.04 $\pm$.00	20 $\pm$ 2%	1.06 $\pm$.01 [1.09]	1.36 $\pm$.01 [1.31]	1.04 $\pm$.00	12 $\pm$ 1%
Core Ultra 9 285K	0.40	1.01 $\pm$.00 [1.01]	1.20 $\pm$.01 [1.13]	1.04 $\pm$.00	20 $\pm$ 2%	1.06 $\pm$.01 [1.09]	1.37 $\pm$.01 [1.32]	1.05 $\pm$.00	12 $\pm$ 1%
RTX 4090, job 112 (higher ratios)									
Ryzen 7 7800X3D	0.57	1.13 $\pm$.01 [1.14]	1.30 $\pm$.01 [1.26]	1.01 $\pm$.01	4 $\pm$ 2%	1.17 $\pm$.01 [1.21]	1.48 $\pm$.01 [1.47]	1.02 $\pm$.00	5 $\pm$ 1%
Core i5-12400	0.77	1.31 $\pm$.01 [1.25]	1.46 $\pm$.01 [1.37]	1.01 $\pm$.00	3 $\pm$ 1%	1.35 $\pm$.01 [1.31]	1.69 $\pm$.02 [1.61]	1.02 $\pm$.00	3 $\pm$ 0%

A MAP OF THE APPENDICES

- Appendix B: the scientific benchmarking checklist, rule by rule.
- Appendix C: the configuration codes and host names used below, and the terms.
- Appendix D: every registered prediction and how it scored, behind Table 1’s evidence column.
- Appendix E: the oracle jobs behind Section 5’s factorial (Fig. 3).
- Appendices F and G: the trace study and the window policies behind Section 7.
- Appendix H: the closed-form account of the time summarised in Section 4, and its registered tests.
- Appendix I: MIN’s schedules machine by machine (Section 5) and the probe’s sensitivity (Section 3).
- Appendix J: the expert cache, the read-time microbenchmark and the system comparison (Section 3).
- Appendix K: the published measurements scored against their bounds (Section 3).
- Appendix L: the artifact.
- The supplement: every clause scored, and three earlier analyses the main text no longer uses: a calibrated time model, the comparison on other GPUs, and an accounting of the gap as a model.

B SCIENTIFIC BENCHMARKING CHECKLIST

How this paper follows the twelve rules of [Hoefer & Belli \(2015\)](#).

Rule	Where and how
1. Report the base case of a speed-up and its absolute performance.	Every ratio in Table 3 is next to both absolute speeds; llama.cpp is configured with per-layer expert offload at equal expert memory, the setting its users run.
2. Explain subsets of benchmarks or resources.	Two models chosen for size and precision (MXFP4, BF16), a third that is FreeToken’s own headline model; budgets span what fits a 32 GB card; the 43.75% budget is omitted where it does not fit. All 30 AIME-25 problems are used.
3. Arithmetic mean for costs, harmonic mean for rates.	The accounting, the panel and the value of foresight average time per token over problems (a cost) and quote speeds as the harmonic mean this implies; Table 3 averages per-problem rates arithmetically, which departs from this rule; every problem decodes the same number of tokens, so the harmonic mean is the right summary, and ratios of total time agree with the reported ratios of mean rates within 0.003.
4. Avoid summarising ratios.	Ratios of means are never averaged across budgets or models; ranges list the per-budget values, and the medians reported across models (Table 16) or across published rows (Appendix K) are medians of per-cell quantities, labelled as such.
5. Report whether measurements are deterministic; give confidence intervals.	Decode is greedy and deterministic per engine; timing is not. Every comparison has a 95% interval over problems; the panel (job 099) adds intervals over machines, the unit that varies most.
6. Do not assume normality.	Intervals over problems are bootstrap percentile intervals (10,000 resamples, paired by problem), with no distributional assumption. Intervals over machines (3 to 15 of them) are <i>t</i> -intervals, which assume the per-machine values are roughly normal; the per-machine values are given beside them (Table 6, the supplement).
7. Compare nondeterministic data soundly.	Paired bootstrap of the ratio of means within a machine; across the panel’s machines, a two-stage bootstrap (machines, then problems) (Kalibera & Jones, 2013); the variant a system is scored with is picked on a separate launch (except the Qwen3 25% row of Table 3).

Table 8. The configuration codes of the appendix tables, the supplement and the scripts, and their names in the main text (Table 2).

Code	Main-text name, and what it is
base	the deployed cache
foa	Dep-1R: the deployed admissions, fetched into the slot in the step
aa	admit every miss
dk	Margin: the deployed cache with a larger admission margin (admitting less)
lrm	the deployed cache ordered by a learned reuse predictor
pf	LA: the deployed cache with the engine’s layer-ahead copy
R1	LA-1R: Dep-1R with the layer-ahead copy (job 109)
R2	LA-1R-margin: LA-1R admitting less (the margin of dk)
fetch	MIN-1R: the greedy schedule, admissions fetched in the step
bypass	MIN-2R: the greedy schedule, served by the CPU, then copied
fetchplan	Few-1R: MIN’s fewest-admission schedule, fetched in the step
bypassplan	Few-2R: the same schedule, served by the CPU, then copied
bypassplanS	Few-2R-early: Few-2R with each copy issued one step earlier (job 112)
lead2	ahead only: MIN’s admissions copied two steps ahead of use, other misses served by the CPU
both2, both3p	read-ahead oracle: MIN-1R plus copies two (three, paced) steps ahead
hitopt, hitoptp	Belady-2R (paced copies)
nb2	Belady-1R, copied two steps ahead
w4, w16	window $W=4, 16$: admit every miss unless the window shows the victim needed sooner
w1, w2	admit every miss, with exact foresight of the next 1 (2) steps
w8r5, allr5	admit every miss, with foresight of 8 steps (the rest of the problem), each future expert seen with probability 0.5
b4, b16	deployed + window $W=4, 16$
b8r5	deployed + window $W=8$, each future expert seen with probability 0.5
learned	Dep-1R with the learned admission and victim order (job 097; not the same as lrm)

Rule	Where and how
8. Check that central tendency is the right summary.	Mean speed over 256 tokens is FreeToken’s own metric; per-token p50 and p99 latencies are recorded for every problem in the released rows.
9. Document all factors and the setup.	Machines, probes, power limits, memory clocks (Section 2), software commits (llama.cpp 4da6337, FreeToken 0d652e7), flags and job scripts are released; Appendix L.
10. Report measurement, synchronisation and summarisation.	One client times every system of Table 3 from streamed token arrivals; the oracle comparisons of Section 5 are timed in process by llama-ec-bench over teacher-forced text, every policy by the same timer (Section 2).
11. Show upper performance bounds.	The bound (Section 3) is a column of Table 3; Table 28 tightens it four ways; Section 3 applies it to the published literature and to our cache with the same procedure.
12. Plot enough to interpret; connect points only for trends.	Figure 3 plots every machine’s ratio with its interval; Fig. 5 connects measured windows for the trend only and marks engine measurements separately; Fig. 8 is a bar chart and Figs. 4 and 9 are scatter plots; the supplement holds the rest.

C NAMES AND TERMS

Table 8 maps the configuration codes used in the appendix tables, the supplement and the scripts to the names of the main text; Table 9 defines the other terms.

D PREDICTION LOG

Each job’s predictions are in the header of its script on the public `gpu` branch, committed before the machine started. Each machine cloned the published branch at boot and ran the header as committed, so git history orders every prediction before its measurement. Of the 92 rentals of jobs 093–112, every commit preceded its rental by at least 2.7 s, and every machine started its job at least 26 s after the push. For seven of them the push and the rental’s creation fall within a second of

Table 9. Terms used throughout.

cell	a model at a budget: the share of each layer’s experts resident on the GPU
E, k, C, L	experts per layer, experts per token, GPU slots per layer, MoE layers
miss	a requested expert not resident: read from host memory by the CPU or by a copy
fetch	copy a miss into a slot and run it there in the same step (one read)
admission	making an expert resident; <i>serve-then-copy</i> : the CPU serves the miss, then a background copy reads it again
prefetch, lead d	copy an expert into a slot at least d steps before its first use (one read)
paced	copies in 16 MB pieces, one in flight
MIN, R^*	Belady’s optimum with bypass on the routing trace; its host reads per token, the fewest of any policy
R_{eng}^*	MIN’s host reads per token in the engine’s oracle: its window ends at the problem’s end, and an expert served in a step stays for that step
1R, 2R	an admitted expert read once (fetched into its slot in the step) / twice (served by the CPU, then copied in the background)
fast / slow link	a link-to-CPU ratio B_p/B_c of at least 0.5 / below 0.5
online / oracle	a policy that sees only past routing / one that reads future routing from a recorded trace
$B_c, B_p, B_{cp}, B_{\text{host}}$	probed host read rates: CPU, link, both, highest by any method
bound	Eq. (1): the speed bound of a machine for a cell
host-/GPU-bound	the bound binds on host reads / on GPU work
$v(F), v(O, F)$	the modelled time with foresight alone / with foresight and overlap (the supplement)
W_{50}	tokens of perfect foresight that close half of an online policy’s excess reads
hosts B, S	the two hosts of the system table (Table 3)
host A	the host of the ablation (Fig. 8)
O1–O6	the hosts of the oracle jobs 093–097 (Table 13, Appendix E)
Pa–Pj	the hosts of the panel, job 099 (Appendix D); later jobs return to Pe, Pf and Pg

each other, and for four GitHub records the push up to 0.8 s after the rental was created (`scripts/reg_timing.py`). Table 10 summarises every job’s clauses, and Table 11 gives the failures. Outcome notes with the statistics are in `prereg/`; the batched-verification variants of Section 7 (rejected drafts free, routed like the true token, or routed at random) are in `prereg/batchk_outcome.md`. Jobs before 073, which built and debugged the engine, also registered predictions in their headers. Their outcome notes are in `prereg/`, and they are not scored here; job 072’s, for one, failed or did not apply on two of its three.

Scoring. Every prediction of jobs 073–112 is split into its separable clauses, and each clause is scored under one rule: *held* when its point estimate is on the predicted side and the 95% paired-bootstrap interval excludes the threshold (for a band, lies inside it); *held (point)* when the point estimate is on the predicted side but no interval exists or the interval crosses the threshold; *failed* when the point estimate is on the wrong side; *untested* when the clause could not be evaluated; and *void* when its job was voided. Jobs 073–098 were scored by hand from the outcome notes, into `prereg/scorecard_clauses.json`, which generates the scorecard. The hand scoring makes one exception: deterministic counts such as reads have no sampling error, so it holds them outright, not held (point). From job 099 on, a script scores each job’s clauses, without that exception.

Two tallies. The hand-scored tally covers the 565 clauses of jobs 073–098: 218 held, 193 held on the point estimate, 142 failed, 8 were untested and 4 void. By era, 7 of 19 scored clauses held in jobs 073–075, when the system was being built; 19 of 23 in jobs 076–081, 11 of them sign-only predictions made after a same-machine pilot; 24 of 38 in jobs 082–087; and 168 of 473 in jobs 088–098. The last row of Table 10 is the overall tally, for jobs 073–112. The hand-scored tally is a subset of it: the table’s rows for jobs 073–098 are the hand scores, and the script-scored jobs 099–112 make up the rest. In the overall row, a deterministic count is therefore held in jobs 073–098 and held (point) after.

Scoring checks. A script (`scripts/scorecard_auto.py`) re-scores every hand-scored clause whose measured value is a number and whose threshold parses, 469 of 565. It agrees with the hand score on 432. Of the 37 differences, 36 are the deterministic counts (reads, divergences, table entries) that the hand scoring holds outright and the script scores held (point), and 1 is a boundary value the hand scoring read unrounded (1.249 against a band starting at 1.25). Of the clauses the script scores held on the point estimate, 213 have no interval and 7 have one that crosses the threshold. The re-scoring is a check; neither tally uses it.

Table 10. Every registered job from 073 on and how its clauses scored: *held* with a 95% interval on the predicted side (or outright, for deterministic counts), held on the point estimate only, failed, and untested or void. Jobs 073–098 were scored by hand, the rest by script; the supplement lists every clause.

Job	What it tested	Held	Point	Failed	Untested
073	ours against FreeToken, same host	3	0	4	0
074	the 40% budget fixed	0	2	4	0
074b	server overhead	2	1	0	1
075	RTX PRO 6000 at 40–60%	2	0	1	0
076	the system table on an RTX 5090	2	1	0	0
077	competitors, gpt-oss	3	0	0	0
078	Qwen3, four systems	3	0	0	0
079	Qwen3 profile	1	3	0	0
080	the fetch split	4	0	0	0
081	the system table with the calibrated fetch split	6	0	0	0
082	held-out prompts and steady state (review run 1)	4	0	3	1
083	gpt-oss, long outputs on hard held-out problems	5	0	0	0
084	all-in-VRAM column and traces (voided: throttled card)	0	0	0	4
084b	all-in-VRAM column and traces, rerun	2	2	1	0
085	a slower PCIe link, gpt-oss	5	0	1	0
085b	a slower PCIe link, Qwen3	5	0	1	0
086	Qwen3.6	1	1	2	0
087	ablation with a static cache	2	1	2	0
088	slow link, fair comparison	5	3	3	0
089	the system table on a second host	9	7	12	0
090	output parity against all-in-VRAM (RTX PRO 6000)	10	0	0	0
091	the grid on an RTX 4090	11	3	5	0
092	the grid on an RTX 3090	6	1	7	1
093	first foresight oracles	9	2	21	4
094	oracles that minimise bytes	12	1	24	1
095	single-read oracles	44	0	12	0
096	the oracle factorial	29	97	12	0
097	the learned admission order	11	53	20	0
098	FreeToken tuned	22	15	7	0
099	the host panel	190	42	39	2
100	windows of foresight	41	122	27	26
101	the crossover by link ratio	6	19	0	0
102	the read-schedule microbenchmark	0	28	5	0
103	fewest-admission schedule (plan not run)	0	4	0	41
104	fewest-admission schedule	1	22	24	0
105	Eq. (4), plain form; layer-ahead copy; fewest-admission set	7	20	4	0
106	Eq. (4) with overlap; admitting less; slow links by round	21	112	48	0
107	Eq. (4) on new machines; admitting less; fewest-admission loads	11	22	13	0
108	Eq. (4) on new desktop-class machines	0	13	6	2
109	the 2×2 and policies without foresight on new machines	47	44	0	0
110	the RTX 5090 trend on RTX 4090s (stopped by V1)	0	0	1	1
111	job 110 relaunched, V1 corrected	24	14	0	0
112	higher-ratio RTX 4090s; timing control; second probe	22	24	12	0
all		588	679	321	84

Failed predictions. Most bands that failed in jobs 073–098 are the sizes of effects whose direction held. In jobs 088–098, 90 of 301 bands failed, as did 23 threshold, 3 equality and 7 sign clauses; 42 of 82 sign clauses held with an interval, and 75 held at least on the point estimate. Table 11 gives the failures job by job: those that matter for jobs 073–098 (the supplement lists all 142), and the families of failures of jobs 099–112.

Table 11: Failed predictions, by job. *Predicted*: the registered band, threshold or sign. Jobs 073–098 were scored by hand, and the table gives the failures that matter; from job 099 on, each job’s line gives its tally as its script scored it. “—”: the reason, where known, is in the job’s outcome note. The supplement lists every clause.

Job	What was predicted	Predicted	Measured	Why it failed, or what it means
<i>Scored by hand, jobs 073–098</i>				
085b	A slower PCIe link, Qwen3. Our lead over FreeToken larger than in Table 3, at 2 or more of 3 budgets (the job’s own restatement of its fourth prediction)	at least 2 of 3	1 of 3	Downgraded from the running log; the pooled 4 of 6 held
088	A slow link, fair comparison. Our lead over FreeToken	1.10–1.50×	1.51–1.58×	Above the band; the lead itself held
089	The system table (Table 3) rerun on a second machine. Our ratio over FreeToken, against host B’s Our speed at the host-bound cells, against host B’s LRU slower than decayed frequency, at the two GPU-heavy cells LRU still ahead of FreeToken at gpt-oss 11%	within ± 0.06 within $\pm 12\%$ 10–30% slower ahead	0.06–0.11 below, at five of six cells 14–17% lower 6–8% slower behind	— — — A sign failure
091, 092	The grid on an RTX 4090 and on an RTX 3090 (the supplement). Our lead over FreeToken; our share of the limit Ours at least $1.8 \times$ llama.cpp, on the slow machines The 4090 machine’s FETCH tables Ours at least $1.3 \times$ llama.cpp on Mixtral The two launch orders agree	10–35% (gpt-oss 11%), 0–15% (Qwen3); 25–45% at least $1.8 \times$ the registered entries at least $1.3 \times$ within 3%	above the bands 1.56–1.78×	FreeToken’s carried-over backend fell far below its band — An equality failure — — 7%; a second difference, 11% at gpt-oss 11% on the 3090 machine, was reported, not scored
093	The first oracle job (Section 5). The hit-optimal oracle faster than the online policy, by 20–80% Its speed at the four host-bound cells Its hit rate and admissions, against the optimum’s The cost of no overlap The cost of all-CPU, Qwen3 The hit-optimal oracle’s gain at the two GPU-bound cells	faster; 20–80% 45–65% of the limit within 3 points; within 30% 5–25% 0–10% below 15%	slower at gpt-oss 11% and Qwen3 12.5%, the two lowest budgets 43.7–44.8% outside, at three and at two of the four cells 0% 15% 48 and 56%	A sign failure as well as a band failure Just below the band — — — —
094	Oracles that minimise bytes: the bands missed as Appendix E describes. The bytes-optimal oracle beats the hit-optimal one, at the two lowest budgets	by at least 15%	by less	—

continued

Where the Seconds Go

Job	What was predicted	Predicted	Measured	Why it failed, or what it means
095	Its 16-token window keeps its gain	at least 70% of it	less, at three cells	—
	Single-read oracles: 44 of 56 clauses held, and the 12 that failed are of size.			
	The model on MIN-1R's counters	within 10%	13–17% off, at four cells	Size
	MIN-1R's reads, against the simulation's	within 10%	17% below, at the two GPU-bound cells	The registered simulator gave a resident carried over from the previous sequence no next use, and so evicted it first; corrected, the simulation is within 4% of the engine's reads at every cell (Appendix E)
	The read-ahead oracle at the host-bound cells	55–85% of the limit	44 and 51% at the two 25% cells	Size
096	The model's over-prediction of the read-ahead oracle	15–50%	7 and 13% at the two lowest budgets	Size
	The paced variant, against the unpaced one	within 10%	12–13% faster at the two 25% cells	Size
	The oracle factorial on O4 and O5: the clauses held at most cells of both machines, and Appendix E lists the failures.			
	MIN-1R's gain, O4 against O5	differs by at most 0.15	by more, at five cells	The largest failure of the job: the difference between the machines
	Belady's single-read prefetch, Qwen3 12.5% on O4	at most 1.05× the online policy	above	—
098	FreeToken tuned (prereg/freetoken_outcome_098.md): its best setting stayed within the predicted 10% of its default, and our ratios within the predicted bands.			
	FreeToken's best backend, gpt-oss	hybrid	offload	An equality failure
	Stock llama.cpp against FreeToken's CPU backend, CPU only (gpt-oss; Qwen3)	at least 2×; at least 1.2×	about equal	—
	The fixed fetch caps; the thread count	within 8% of the calibrated split; 8 threads within 3% of the default or slower	outside	As the job's outcome note describes
	<i>Scored by script, jobs 099–112</i>			
099	The host panel (scripts/panel_099.py; prereg/scorecard_099.json, prereg/panel_outcome_099.md): 273 clauses; 190 held, 42 held (point), 39 failed, 2 untested.			
	Bands set from four earlier machines, none of them link-starved	the registered bands	most failures, on the two machines whose link reads at less than half their CPU's rate	The bands did not allow for dependence on the machine
	The planning time of the noisy whole-future window	at most 150 μs per step	above	—
	Windows of foresight (scripts/job100.py; prereg/window_outcome_100.md): 216 clauses; 41 held, 122 held (point), 27 failed, 26 untested.			
	The deployed-path windows miss within 10% as often as a replay that admits at once	within 10%	more misses; most of the job's failures	Their copies land late
100	The probe-only model, on the half-width-link machine	within 12%	outside	—
	Four other bands or orderings	as registered	at their edges	—
	The crossover by link ratio (scripts/job101.py; prereg/crossover_outcome_101.md): 25 clauses; 6 held, 19 held (point), 0 failed, 0 untested.			
	The read-schedule microbenchmark on three machines (scripts/job102.py; prereg/readsched_outcome_102.md): 33 clauses; 0 held, 28 held (point; none has an interval), 5 failed, 0 untested.			
	Waiting per layer costs time against the analytic schedule	at least 0.03 of the host term	nothing measurable on two of the three machines	Four of the five failures

continued

Where the Seconds Go

Job	What was predicted	Predicted	Measured	Why it failed, or what it means
	The order of the two budgets, Intel machine	as registered	the other way	—
103	MIN’s fewest-admission schedule against the greedy one; the plan did not run (<code>scripts/job103.py</code>): 45 clauses; 4 held (point); the relaunch of Pf, 41 untested.			
104	Job 103 repeated, with the plan run (<code>prereg/minadm.outcome.104.md</code>): 47 clauses; 1 held, 22 held (point), 24 failed.			
	Few-1R’s copies and misses at gpt-oss 25%, against MIN-1R’s	copies at most 0.70×; misses within 3%	0.73×; 5.3% more	—
	The engine’s misses, against the machine’s replay	within 2%	more	—
	Few-2R’s reads, against MIN-2R’s	at most 0.90×	more	—
	The 2×2’s interaction, with the plan’s set	shrinks	grew	—
	Few-2R against MIN-2R	at most 0.01 behind	0.047 behind, on Pg at gpt-oss 25%	Held at the other 5 of 6 cells
105	The plain form of Eq. (4) with G profiled on each machine, LA and Few-1R (<code>scripts/job105.py</code> ; <code>prereg/sumlaw.outcome.105.md</code>): 31 clauses; 7 held, 20 held (point), 4 failed. Every failure is the relation at gpt-oss 25%.			
	The relation at gpt-oss 25%, per machine	within 6%	over-predicts the deployed cache by 6.9–9.3% on 3 of four machines (3.8% on Pf)	At 11% it is within –1.5 to 4.7%
	Its pooled median error	at most 4%	4.2%	The 25% cells take it past the band
106	The relation of Eq. (4) with its overlap term on both models at two budgets, the two online admission rules, and MIN’s sets launch by launch, on five machines (<code>scripts/job106.py</code> ; <code>prereg/onlineadmit.outcome.106.md</code>): 181 clauses; 21 held, 112 held (point), 48 failed: 36 on the two unstable machines, 11 on the 3 stable ones and 1 pooled. Clauses on the two server machines new to the job	as registered	36 failed	One computed wrong outputs; the other varied by more than the registered 2% between rounds (Appendix I)
	Margin’s speed-up at gpt-oss	at least 1.01	below, on two stable machines	—
	The learned order’s speed-up	at least 1.00	below, on the fast-memory machine	—
	The learned order’s host time	at most 400 μ s (gpt-oss), 700 μ s (Qwen3) per step	above, on the fastest-link machine	—
	The relation’s prediction of both rules’ ratios, Pf at 25%	within 0.03	off by 0.030 and 0.045	—
	The relation’s median error over all machines	at most 4%	above	—
107	The relation, Margin and MIN’s fewest-admission set loaded both ways, on machines never rented before (<code>scripts/job107.py</code> ; <code>prereg/newhosts.outcome.107.md</code>): on its two valid machines, 46 clauses; 11 held, 22 held (point), 13 failed.			
	The relation, per cell	within 6%	beyond, at 7 of 8 cells	—
	The relation’s pooled median error; the overlap form against the plain one	at most 4%; overlap form better	above; not better	—
	Margin’s speed-up	at least 0.995	below, at one cell	—
	The relation’s prediction of Margin’s speed, on the server	within 0.03	outside, at two cells	—
	Valid machines	at least three	two	Inconclusive by its registered rule
108	The relation on machines never rented before, gpt-oss only (<code>scripts/job108.py</code> ; <code>prereg/smallhosts.outcome.108.md</code>): 21 clauses; 13 held (point), 6 failed, and the rest could not be tested.			
	The relation on the EPYC 7543, both budgets	within 8%	outside	—
	Its implied read rate, both budgets	0.85–1.25 of B_{host}	outside	—

continued

Where the Seconds Go

Job	What was predicted	Predicted	Measured	Why it failed, or what it means
	Pooled: the cells within 6%; the median error	at most one cell beyond 6%; at most 4%	2 of 6 within 6%; 6.7%	—
109	The 2×2 of Section 4, the read-ahead oracle and four variants without foresight, on desktop-class machines never rented before (<code>scripts/job109.py</code> ; Table 6): 91 clauses; 47 held with a 95% interval over problems, 44 held (point), 0 failed. Of its 11 predictions 11 held; one clause of P8, for ratios of at least 0.9, had no machine to test it.			
110	The RTX 5090 trend of Section 6.2, frozen in its header, on RTX 4090 machines: Valid machines	at least 3	none	void as registered. The output check used the RTX 5090’s reference loss
111	Job 110 relaunched with the RTX 4090’s own reference: job 110’s clauses on three valid machines; 24 held with an interval, 14 held (point), 0 failed. Its sixth prediction, for machines with a ratio of at least 0.5, had none to test.			
112	The second card at higher ratios, the timing control, a second probe and two relaunches of job 109’s machines (<code>scripts/job112.py</code> ; Tables 5 and 6): 58 clauses; 22 held with an interval, 24 held (point), 12 failed.			
	Few-2R-early’s misses	at most 0.95 of Few-2R’s	0.97–0.98 at 11%, 0.95 at 25%; failed on 5 of 8 machine-budgets	The replay of the publication rule predicted 0.91; the engine’s fetch table keeps MIN’s set from forming
	Few-2R-early no slower than Few-2R	at least –0.01	–0.05–0.03 at 11%, –0.03–0.01 at 25%; failed on 7 of 8	The late landing is not what holds the two-read arm back

Changes against the running log. Against the running log kept during the study, two clauses are downgraded. A `llama.cpp` clause of job 074b that the log counted as held was never run, and job 085b’s own restatement of its fourth prediction, “at least 2 of 3”, failed at 1 of 3, though the pooled 4 of 6 held. Job 073’s “identical outputs” is scored on fully identical outputs (0–3%) rather than on agreement of the first 160 characters (27–47%). Ten clauses the log counted as held, among them job 087’s first, hold on the point estimate only under the interval rule. The file `prereg/scorecardOutcome.md` lists every change.

Reruns and rentals without results. Job 084 ran on a throttled card and was rerun, with its predictions unchanged, as 084b/084c. Job 085’s Qwen3 half ran as 085b on a second machine with a slow link after a hung conversion. Rentals that produced no results were destroyed and are in the rental ledger (`gpu/vast_ledger.json`): two of job 097’s four rentals; the first two for 099a and the first for 099d, replaced before any measurement; 100d, which stalled downloading the model; 100e, which never reached its job; 103c, which never started its job; and 105c, 105d and 105g below.

Jobs 099–104. Job 099’s two untested clauses are one machine’s replay check, whose step failed when a Python package did not install. Job 100’s header gained a ninth prediction, about the two replacement machines, after the first four machines had started; it compares the replacements with values one of which was already measured. Job 100’s untested clauses are the planning time of the deployed-path windows, which the engine does not time, and the paired prediction for a machine that never reached a timed step. Job 101 relaunched the lowest-ratio panel machine and added a second Ryzen 7 9800X3D behind a slower link. In job 103 the machines could not install the plan’s solver beside Debian’s `numpy`, so the engine ran the greedy schedule in the plan configurations, as it logs. Job 104 repeated job 103 with the plan in its own environment on three of the same machines: Pf; a second Core Ultra 9 285K, whose link reads at 0.49 of its CPU rate; and Pg. Job 103’s 5950X had turned out to be Pg’s second launch. Most of job 104’s speed predictions held: Few-1R gains on Pf, and gains at least 0.03 over MIN-1R at gpt-oss 11% on every machine.

Jobs 105 and 106. Two of job 105’s clauses predicted quantities already measured: the profiled G (job 069c) and the plan’s copies, which are a property of the trace (job 104). They test that the engine reproduces them. Every speed prediction of job 105 held: LA gains at least 3% on the machine with a ratio above 0.8 and loses on both below 0.4, and Few-1R beats the deployed cache on every machine, beats MIN-1R by at least 0.03 on the three with a ratio under 0.7, and copies at most 0.65 of MIN-1R’s experts. Three more launched machines produced no results: 105c never started its job, and the job’s bandwidth gate stopped 105d and 105g, whose cards were throttled, before any timed run. Job 106 ran on five machines. Two server machines were new to the job; one computed wrong outputs, and the other varied by more than the registered 2% between rounds (Appendix I). The 3 stable machines are Pf, the Threadripper 9960X of job 105 and panel machine Pe, all rented before, and on them the relation held on every cell. Margin’s pooled median speed-up of at least 1.02 held only because of the two unstable machines; on the stable ones it is 1.01.

Jobs 107 and 108. Job 107 had validity gates registered before launch. Of 7 machines rented, 3 stopped at the gate and 2 failed the round check. The test had two valid machines of the three it required, which by its registered rule makes it inconclusive; its per-cell clauses failed on both. The profiled GPU compute, Margin's reads and pooled median, both fewest-admission loads and the crossover between them held. Replacing machines that failed the round check, and the rules that chose the last two machines, were amendments committed before each machine started. Jobs 102, 105 and 106 also had their headers edited after launch, each time to substitute a machine, with the predictions unchanged and marked so in the header. Job 108 tested the relation on machines never rented before, with the class drawn after job 107 gated before any download, gpt-oss only, and an 8% band. Of 6 machines rented, 2 stopped at the gate, 1 failed the round check, and three were valid. The profiled G of the EPYC 7543 could not be tested.

Job 109. The job fixed its population, machines never rented before with a link-to-CPU ratio of at least 0.5, before any started; five machines started, and all five passed every gate and round check. A smoke run on gpt-oss-20b, on a machine outside the sample, first checked that the two new combinations run and compute the same model. One listed offer was no longer offered and was skipped as registered. The job's deadline cut the 25% round of the Ryzen 9 5950X, the slowest machine. The header's statement that no listed offer was in the rental ledger was wrong for one: the Core i9-13900KF's offer had been rented twice before (the first rental for 099d, and 100e), both times without a result. Its GPU had never been measured, and it passed the identity gate. Several of the job's bands were wide against what earlier jobs had shown: the mean share closed by MIN-1R (0.25 to 0.55, where the panel had shown 33%), the margin's speed (0.98 to 1.06, where it had run 1.020 $\times$ the deployed cache) and the capture bound (at most 0.35). These held, but could hardly have failed. The clauses that could have failed are the interaction's sign on every machine and the per-machine bounds on the variants without foresight; job 110's predictions 3 to 5 are range checks of the same kind.

Jobs 110 and 111. Job 110 tested the RTX 5090 trend of Section 6.2, frozen in its header, on RTX 4090 machines; five machines started. Of these, one stopped at its ratio gate (at 0.2499, below 0.25), and every other one failed its output check after one round. That check used the RTX 5090's reference loss. The RTX 4090 computes a loss 3% higher in every configuration of every process, so the check failed by design rather than by fault, and job 110 is void as registered. Job 111 relaunched three of the four machines that had run a round, with the card's own reference (its first round's loss on one of them) and job 110's predictions unchanged. The fourth, 110e, was not relaunched: its GPU was left out of the relaunch list, an oversight we found after the job. The predictions were committed before any RTX 4090 data, and the corrected reference after one round on these machines. Of job 111's first launches, two failed to download the model, before any model run; they were relaunched once, by an amendment committed before the relaunch.

Job 112. The last job's predictions were committed before any of its machines started. It asked whether the RTX 5090 trend carries to RTX 4090 machines at higher ratios (job 110's predictions unchanged, the ratio gate at 0.45); whether landing the two-read arm's copies in time rescues it (**Few-2R-early**, a new plan file for an engine path that had run before, so no smoke run); whether a second probe after the timed runs agrees with the first; and whether two of job 109's RTX 5090 machines reproduce on a second rental. One of those two, job 109d's machine, never finished downloading the model in two hours and was destroyed without results; job 109f's machine replaced it, started while the download was still running, once two rounds could no longer fit before its deadline. The timing predictions came from a replay of the engine's publication rule; in the engine the early copies barely changed the misses (0.97–0.98 of **Few-2R**'s at 11%, against a registered 0.95 or less) and slowed the arm by up to 0.05 of the deployed cache's speed (against a registered 0.01): the first clause failed on 5 of 8 machine-budgets and the second on 7 of 8. The second read's cost exceeded the late landing's, and **Few-2R-early** stayed at or below 1.10 $\times$, as registered. The first probe ran while the model downloaded on some machines and after it on others; the second ran on the idle machine. On the two relaunched RTX 5090 machines the deployed cache's time came within 0.7% of job 109's, while the probe's link-to-CPU ratio moved by up to 12% between the two rentals.

The clause-by-clause tables (565 rows for jobs 073–098, 273 for job 099, 216 for job 100, 25 for job 101, 33 for job 102, 45 and 47 for jobs 103 and 104, 31 for job 105, 181 for job 106, 46 for job 107, 21 for job 108, 91 for job 109, 2 for job 110, 38 for job 111, 58 for job 112) are in the supplement ([paper/supplement.pdf](#)), generated by the same scripts.

E FORESIGHT IN THE ENGINE

Four jobs ran oracle policies in the engine, each on a rented RTX 5090 machine with weaker host memory than host B's. Jobs 093–097 report ratios of mean speeds rather than of mean times per token; recomputed either way, no ratio moves

Table 12. The single-read oracles (job 095; RTX 5090, Ryzen 9 9950X, host memory 61.5 GB/s at best) beside Belady-2R. Per cell: the deployed cache’s speed and its share of this machine’s bound; for each oracle with the whole sequence in view, its ratio to the deployed cache (paired, 95% interval), hit rate, host reads per token (CPU misses + fetches + admissions) as a multiple of R^* (each read counted once), share of the bound and the calibrated model on its own counters against its measured time. MIN-1R: MIN with bypass, admitted misses fetched into their slot this step; *ahead only*: MIN’s admissions copied when their first use is at least two steps away, other misses served by the CPU; *read-ahead*: MIN-1R and ahead only together; *read-ahead, paced*: the same on the paced copy path with a three-step lead (the read-ahead oracle of Table 2); Belady-2R: Belady’s rule without bypass, unpaced. The simulation columns give the trace’s prediction of reads and hits for MIN-1R (its reads are R_{eng}^* , Appendix E) and the read-ahead oracle.

Cell	deployed			MIN-1R							read-ahead							Belady-2R							ahead only		read-ahead, paced		R_{eng}^*	sim. MIN-1R		sim. read-ahead	
	tok/s	% bound	ratio	hits	reads/ R^*	% bound	model	ratio	hits	reads/ R^*	% bound	ratio	hits	reads/ R^*	% bound	ratio	% bound	ratio	% bound	ratio	% bound	hits	reads	hits	reads	hits	reads						
gpt-oss 11%	55.5	46	1.29 [1.28, 1.29]	72	1.04	59	-8%	1.39 [1.39, 1.40]	86	1.23	64	0.91 [0.90, 0.92]	98	1.77	42	1.26	58	1.45	67	39	73	46	87										
gpt-oss 25%	90.3	30	1.32 [1.31, 1.33]	89	1.08	39	-13%	1.49 [1.48, 1.50]	97	1.45	44	1.14 [1.12, 1.15]	99	2.11	34	1.45	43	1.66	50	16	89	22	99										
gpt-oss 40%	128.8	25	1.25 [1.24, 1.27]	95	1.14	32	-14%	1.39 [1.37, 1.41]	99	1.60	35	1.27 [1.25, 1.28]	99	2.20	32	1.38	35	1.49	38	7	95	12	100										
Qwen3 12.5%	32.3	48	1.34 [1.34, 1.35]	74	1.04	64	-9%	1.42 [1.41, 1.43]	84	1.20	68	0.69 [0.68, 0.71]	77	2.47	33	1.22	58	1.41	68	100	74	115	84										
Qwen3 25%	51.8	34	1.38 [1.37, 1.39]	88	1.04	48	-15%	1.48 [1.46, 1.49]	96	1.42	51	1.06 [1.05, 1.08]	99	2.00	37	1.41	48	1.67	57	45	88	61	97										
Qwen3 43.75%	91.4	30	1.26 [1.25, 1.28]	96	1.08	38	-17%	1.41 [1.39, 1.42]	99	1.62	42	1.27 [1.26, 1.29]	100	2.29	38	1.40	42	1.50	45	15	96	23	100										

by more than 0.006. The fourth (096) is the confirmatory factorial of Fig. 3: ten configurations per cell, each in a fresh context, in an order shuffled per cell with a recorded seed, on hosts O4 (Ryzen 9 9950X: 43, 45 and 51 GB/s by the CPU, over the link and at best) and O5 (Ryzen 9 9950X3D: 47, 27 and 51 GB/s), with ten predictions per machine committed before either started (`prereg/foresight_outcome_096.md`). They held at most cells. The failures: MIN-1R’s gain above the predicted band at Qwen3 25% on O4 and different between the two machines by up to 0.26 against the 0.15 predicted; Belady-1R faster than predicted at Qwen3 12.5% on O4, with reads below the predicted 1.25–1.9 $\times$ those of the *ahead-only* oracle (below) at Qwen3 12.5% on both machines and at gpt-oss 11% on O4 (down to 1.09 $\times$); the bytes share at Qwen3 12.5% on O4 (60% of the gap against a predicted 25–55%); and the reads Dep-1R saves at gpt-oss 40% on O5, just above the predicted 22%. Dep-1R came to 0.97–1.03 $\times$ the deployed cache at the host-bound cells, at the bottom of the predicted 0.97–1.12 $\times$ although its reads fell as predicted, and to 0.95–1.02 $\times$ at the GPU-bound cells. Table 12 lists every cell of the single-read factorial of Section 5 on host O3 (a Ryzen 9 9950X whose probe reads 53 GB/s by the CPU at the helper count, 43 over the link, 60 with both and 61.5 at best; the model’s FETCH tables 0, 0, 1, 2, 2 for gpt-oss and 0, 0, 1, 1, 2, 3, 4, 4, 5 for Qwen3), and Table 13 the milliseconds per token of the engine states the three jobs realised beside each machine’s bound and the model’s values. Table 14 lists the first job (093), Belady-2R with a window sweep on host O1 (a Ryzen 9 9950X: 44, 46, 49 and 50.5 GB/s; tables 0, 0, 1, 2, 3 and 0, 0, 1, 2, 3, 4, 5, 6, 7), and Table 15 the second (094), MIN-2R beside Belady-2R on host O2 (a Ryzen 9 9950X: 44, 46, 50 and 50.6 GB/s; the same tables). On O1 and O2 every cell but gpt-oss 40% and Qwen3 43.75% is host-bound in Eq. (1); on O3, whose memory is stronger, the same four.

The single-read oracles. In MIN-1R the step’s plan is computed before the layer runs from the trace’s record of the current step: the misses MIN with bypass would admit are fetched by the engine’s FETCH operation into their victims’ slots and run from there, and every other miss runs on the CPU; the oracle refuses an admission when no resident’s next use is further than the candidate’s. Reading ahead admits, each step, the non-resident experts whose first use is at least d steps away ($d=2$ unpaced, 3 paced), provided the victim’s next use is further than the candidate’s use after that first one; an expert used sooner than d steps away misses on the CPU (the *ahead-only* oracle), or is fetched as in MIN-1R (the read-ahead oracle; paced with $d=3$, the form of Table 2). The same policies simulated on the trace with the engine’s semantics (the lookahead within the sequence, residents carried over with their next use recomputed at each sequence start) predict the engine’s reads and hit rates: MIN-1R’s hit rate within 0.4 points and its reads within 0 to 4%, the read-ahead oracle’s hits within two points (Table 12). Simulated, MIN-1R reads R_{eng}^* experts per token, MIN’s count in the engine: its window ends at the problem’s end, and an expert served in a step stays for that step. The simulator the job’s predictions were registered against gave a resident carried over from the previous sequence no next use, so it was evicted first; that over-counted MIN-1R’s reads by 17% at the two GPU-bound cells, which is why two of its reads clauses failed (Appendix D). Each configuration of llama-ec-bench starts with a cold cache in a fresh context. The paced copy path copies one 16 MB piece at a time on a single copier thread and publishes a landed copy at the start of the next step; the unpaced path copies each admission as one transfer. The background admissions of Belady-2R drive host memory at 71–82% of the probe’s best rate at the host-bound cells, the single-read oracles that read ahead at 64–85% (the paced read-ahead oracle 71–82%), the deployed cache itself at 66–82%, MIN-1R at 42–67%. The predictions and their outcomes are in `prereg/foresight_outcome_095.md`: of 56 clauses, 44 held and 12 failed, all of size (Appendix D).

Table 13. The accounting, measured: milliseconds per token (1000 over the mean speed; the main text averages times per token, which gives values up to 1.3% higher on these runs) of the engine’s states that the oracle jobs realised, on each oracle machine, beside that machine’s bound and the model’s values for the states it prices; O1–O3 ran jobs 093–095 in a fixed order, O4 and O5 job 096 in a shuffled order. *Deployed*: the deployed cache; *Dep-1R*: the same with each admission fetched into its slot in the step that misses it (one read, no foresight); *no ovl.*: the deployed cache with its within-layer overlap off; *all CPU*: with no FETCH table; *MIN-2R*: MIN with bypass with background admissions (two reads per admitted expert); *Belady-2R*: Belady’s rule without bypass, which admits what MIN bypasses, unpaced and on the paced copy path (O1 and O2 ran only the paced form); *Belady-1R*: Belady’s rule within a two-step lead with each admission read once; *MIN-1R*: MIN with bypass, admitted misses fetched into their slot (one read, serialised: the model’s foresight-only state $v(F)$); *read-ahead*: MIN-1R with MIN’s admissions also copied two steps ahead (one read, overlapped: the model’s $v(O, F)$), unpaced and on the paced copy path with a three-step lead (the read-ahead oracle of Table 2); *bound*: Eq. (1) on that machine; $v(F)$, $v(O, F)$, $v(\text{all})$: the model.

Host	Cell	Deployed	Dep-1R	no ovl.	all CPU	MIN-2R	Belady-2R unpaced	Belady-2R paced	Belady-1R	MIN-1R	read-ahead unpaced	read-ahead paced	bound	$v(F)$	$v(O, F)$	$v(\text{all})$
O1	gpt-oss 11%	21.12	–	21.01	22.91	–	–	22.88	–	–	–	–	10.06	14.54	11.27	10.06
O1	gpt-oss 25%	12.89	–	12.91	14.21	–	–	9.22	–	–	–	–	4.03	8.04	5.10	4.03
O1	gpt-oss 40%	8.90	–	8.95	–	–	–	5.99	–	–	–	–	1.97	5.75	3.84	1.97
O1	Qwen3 12.5%	36.92	–	36.91	43.29	–	–	40.29	–	–	–	–	18.04	24.90	19.29	18.04
O1	Qwen3 25%	22.73	–	22.76	26.60	–	–	18.22	–	–	–	–	8.11	14.04	9.22	8.11
O1	Qwen3 43.75%	12.71	–	12.67	–	–	–	8.12	–	–	–	–	3.32	8.25	5.51	3.32
O2	gpt-oss 11%	20.85	–	–	20.17	–	–	21.57	–	–	–	–	10.04	14.27	11.96	10.04
O2	gpt-oss 25%	12.72	–	–	10.77	–	–	7.80	–	–	–	–	4.02	7.86	5.86	4.02
O2	gpt-oss 40%	8.79	–	–	7.13	–	–	5.62	–	–	–	–	1.97	5.63	3.78	1.97
O2	Qwen3 12.5%	36.64	–	–	36.58	–	–	38.74	–	–	–	–	18.00	24.70	20.06	18.00
O2	Qwen3 25%	22.24	–	–	19.67	–	–	15.80	–	–	–	–	8.09	13.58	9.90	8.09
O2	Qwen3 43.75%	12.02	–	–	10.11	–	–	7.42	–	–	–	–	3.32	7.56	4.90	3.32
O3	gpt-oss 11%	18.01	–	–	–	–	19.79	–	–	14.00	12.91	12.39	8.26	12.63	9.49	8.26
O3	gpt-oss 25%	11.08	–	–	–	–	9.74	–	–	8.42	7.44	6.66	3.31	7.05	4.28	3.31
O3	gpt-oss 40%	7.77	–	–	–	–	6.14	–	–	6.20	5.58	5.20	1.95	5.24	3.56	1.95
O3	Qwen3 12.5%	30.95	–	–	–	–	44.54	–	–	23.02	21.78	21.89	14.81	20.35	16.06	14.81
O3	Qwen3 25%	19.32	–	–	–	–	18.17	–	–	14.01	13.09	11.58	6.66	11.68	7.76	6.66
O3	Qwen3 43.75%	10.94	–	–	–	–	8.59	–	–	8.66	7.78	7.28	3.30	7.24	4.76	3.30
O4	gpt-oss 11%	20.59	20.18	–	–	20.11	19.78	22.03	19.64	15.13	14.06	13.78	10.02	13.77	11.63	10.02
O4	gpt-oss 25%	12.43	12.07	–	–	10.55	9.76	8.13	9.61	8.72	7.65	6.87	4.01	7.40	5.34	4.01
O4	gpt-oss 40%	8.39	8.19	–	–	6.91	6.15	5.38	6.03	6.28	5.59	5.21	1.97	5.12	3.20	1.97
O4	Qwen3 12.5%	36.36	35.83	–	–	36.54	48.80	39.66	31.64	25.29	24.01	24.58	17.97	23.98	19.88	17.97
O4	Qwen3 25%	22.09	21.75	–	–	20.06	17.74	16.93	17.57	14.60	13.50	12.24	8.08	13.12	9.54	8.08
O4	Qwen3 43.75%	11.94	11.74	–	–	10.09	8.44	7.17	8.33	8.69	7.72	7.29	3.32	7.32	4.56	3.32
O5	gpt-oss 11%	21.26	21.31	–	–	21.47	33.34	25.57	33.03	18.34	17.97	16.39	9.94	15.22	11.20	9.94
O5	gpt-oss 25%	13.01	13.35	–	–	11.18	15.87	11.17	15.60	10.60	10.54	8.68	3.98	9.13	5.06	3.98
O5	gpt-oss 40%	8.73	9.22	–	–	7.16	8.34	6.25	8.13	7.27	6.87	5.91	1.97	6.26	3.62	1.97
O5	Qwen3 12.5%	37.63	37.42	–	–	40.11	69.42	44.89	49.11	31.37	31.27	30.01	17.82	27.95	19.27	17.82
O5	Qwen3 25%	23.24	23.27	–	–	21.12	30.78	21.76	30.47	18.51	19.64	16.03	8.01	16.77	9.15	8.01
O5	Qwen3 43.75%	12.49	12.92	–	–	10.32	12.08	9.00	11.86	10.21	9.61	8.23	3.31	8.94	5.02	3.31

The double-reading oracles. Belady-2R and MIN-2R read the routing of the next W decode steps; their copies ride the paced admission path, and an expert still in flight at its step runs on the CPU like any other miss. Belady-2R, the *hit-optimal* oracle, admits the experts those steps will route to, soonest first, evicting the resident whose next use is furthest (Belady within the window), for one future use as readily as for ten. MIN-2R, the *bytes-optimal* oracle, is MIN with bypass with its admissions made through the background path: a miss is served where it is, by the CPU or a demand copy, and admitted only if its next use comes before the furthest next use among the residents. With the whole sequence in view Belady-2R runs $1.40 \times [1.38, 1.41]$ the deployed cache at gpt-oss 25% and $1.25 \times [1.24, 1.26]$ at Qwen3 25% on host O1, $1.63 \times$ and $1.41 \times$ on O2, and $1.48\text{--}1.62 \times$ at the two GPU-bound cells; half of its gain at gpt-oss 25% arrives between $W=4$ and 16 (9.2 tokens by log interpolation; Qwen3 25%: 6.1 against the trace’s 3.9). Where the host binds hardest it loses, 3–8% at gpt-oss 11% and Qwen3 12.5% on both machines, and at every window on O1 ($0.78\text{--}0.92 \times$): it reads more host bytes than the deployed cache (82 and 195 experts per token against 63 and 160), $2.02\text{--}2.18 \times R^*$, because Belady admits an expert used once where the optimum bypasses it and re-copies what it evicts (its admissions, each one read, are most of its reads; a miss served on the CPU before its copy lands is the second read of that expert). An unpaced copy path, run at the four host-bound cells of O1, reaches 98–99% hits at three of them and still loses at gpt-oss 11% and Qwen3 12.5% (where it hits 75%), so copies that arrive in time do not remove the loss there: the extra bytes cause it. MIN-2R gains 13–23% at the four cells of 25% and above and 0–3% at the two below (6% with a 16-token window); its misses fall (30 to 20 and 85 to 64 per token at 25%) but its total reads, misses plus the admissions that read each admitted expert again, are $1.73\text{--}2.04 \times R^*$ and exceed the deployed cache’s by 4–5% at the two lowest budgets; the model on its own counters predicts its time within 9% at every cell, and the bytes it saves account for 4–21 points of its 13–23% gain at the four cells (most of it at gpt-oss 40%

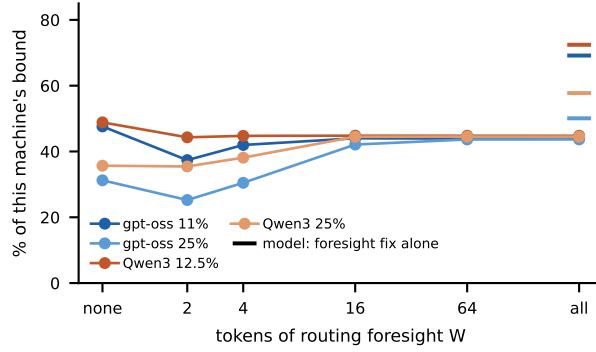


Figure 6. The first oracle job (093, host O1): speed as a percentage of this machine’s bound against the window W of future routing Belady-2R sees, at the four host-bound cells. “none” is the deployed cache; “all” the whole sequence; points are connected for the trend only. The tick at the right is the model’s foresight-only term for the cell, the speed it predicts with that one fix; the oracle’s host traffic sits at 89–94% of the probe’s best rate at every window, so its speed is R^* over its reads times that utilisation, 44–45% of the bound with the whole sequence in view.

and Qwen3 43.75%, a third at Qwen3 25%). Belady-2R reads $1.01\text{--}1.13\times$ the host bytes of MIN-2R on O2 and gains 41–63% at the same four cells, and the model, with its GPU term calibrated on the deployed cache’s run, over-predicts those runs’ time per token by 37–53% (on O1, 24–39%): its gain is the overlap of copies issued ahead of their use with the step, which the model charges in sequence. Pacing the deployed cache’s own copies changes it by under 1% ($0.997\text{--}1.005\times$). With no FETCH table (every miss on the CPU) the deployed cache runs 8–15% slower at the four host-bound cells; with the engine’s within-layer overlap of a layer’s CPU misses with its GPU hits turned off it changes by under 1% at every cell (0.5% faster to 0.6% slower), so the overlap the accounting charges is across the step and in rebalancing, not within a layer. MIN-2R with a 16-token window keeps 34–176% of the whole-sequence gain at the five cells where that gain’s interval excludes zero (more than all of it at gpt-oss 11%, where the shorter window admits less).

Why the misses of MIN-2R stay above R^* . Besides the second read, its misses alone are $1.31\text{--}1.47\times R^*$ at the four host-bound cells (at the two GPU-bound ones the ideal simulation below is itself 6–10% above R^* , which sees across sequences). The engine’s paced copy path moves one 16 MB piece at a time from pageable host memory on a single copier thread, behind the demand copies, publishes a landed copy at the start of the next step, and admits nothing while 64 copies are queued; at the 16–30 admissions per token (210–285 MB) of the two lowest budgets it lags the admissions by steps, and an expert still in flight is a miss the CPU reads again. The last four columns of Table 15 simulate MIN-2R on the measured routing trace with the engine’s own semantics (the lookahead within the sequence, residents carried over, no queue cap) and a copy that lands d steps after it is issued, during which the expert misses and is not re-admitted: at $d=1$, the ideal, the misses come within 0–4% of R^* at the host-bound cells; the engine’s misses meet the simulation at $d=2.8\text{--}4.7$ there and at 3.0–3.5 at the GPU-bound cells, three to five steps. At the fitted latency the simulation admits $1.1\text{--}1.9\times$ what the engine admitted, so the queue cap, not only the latency, keeps the engine’s misses above the ideal. The predictions and their outcomes are in `prereg/foresight_outcome_093.md` and `094.md`: of 36 clauses for the first job, 9 held, 2 held on the point estimate, 21 failed and 4 were untested; of 38 for the second, 12 held, 1 on the point estimate, 24 failed and 1 was untested (Appendix D). The second job’s failures are of size, not direction: the misses of MIN-2R were predicted within 15% of R^* and came 29–47% above (the copy latency and the queue cap); its hit rate was predicted within 4 points of the optimum’s and fell 5–11 points short at three cells; its gain was predicted at 10–45% at the host-bound cells and came below that at the two lowest budgets; the model was predicted to over-predict it by 10–40% at the host-bound cells and over-predicted by 5–9%; and the two oracles were predicted within 15% of each other at 25% and above, where Belady-2R was $1.24\text{--}1.38\times$ faster.

Where a token’s time goes at a GPU-bound budget. On Qwen3 at 43.75% Nsight Systems splits a token of our cache (FreeToken in parentheses) into 4.93 (5.20) ms of expert copies over PCIe, 2.29 (2.21) of resident-expert products, 1.65 (1.34) of attention projections, 0.49 (0.37) of attention, 0.38 of output head and 0.97 (0.40) of norms, small kernels and cache control, with neither system overlapping its copies with GPU compute. Turning off the within-layer overlap of a

Table 14. Belady-2R, the hit-optimal oracle, in the engine (job 093; RTX 5090, Ryzen 9 9950X, host memory 50.5 GB/s at best). tok/s of the deployed cache and of the oracle with the whole sequence in view; their ratio, paired by sequence, with its 95% interval; hit rate and misses per token of each (experts read from host memory by the CPU or by a demand copy), the optimum’s hit rate and reads (R^*) in parentheses, and the oracle’s background admissions (Belady’s rule without bypass: it admits what MIN bypasses); speed as a percentage of this machine’s bound; the accounting’s foresight-only term $v(F)$ for the cell in tok/s; the share of that term the oracle recovered and the share of the gap to the bound it closed.

Cell	deployed	oracle	ratio	hits % (opt.)	misses/token (opt.)	oracle adm.	% of bound	$v(F)$	recovered %	closed %
gpt-oss 11%	47.4	43.7	0.92 [0.91, 0.93]	58 → 67 (73)	60 → 47 (38)	35	47.6 → 44.0	68.8	−27	−16
gpt-oss 25%	77.6	108.5	1.40 [1.38, 1.41]	79 → 92 (89)	30 → 11 (15)	20	31.2 → 43.7	124.4	76	41
gpt-oss 40%	112.4	166.9	1.48 [1.46, 1.51]	89 → 98 (95)	15 → 3 (7)	12	22.1 → 32.8	174.0	92	42
Qwen3 12.5%	27.1	24.8	0.92 [0.91, 0.92]	60 → 66 (75)	155 → 132 (97)	63	48.9 → 44.8	40.2	−28	−18
Qwen3 25%	44.0	54.9	1.25 [1.24, 1.26]	78 → 89 (89)	85 → 42 (43)	46	35.7 → 44.5	71.2	52	31
Qwen3 43.75%	78.7	123.1	1.56 [1.53, 1.59]	92 → 99 (96)	32 → 5 (14)	25	26.1 → 40.8	121.3	103	49

Table 15. MIN-2R, the bytes-optimal oracle, beside Belady-2R, the hit-optimal one, on the second oracle machine (job 094; RTX 5090, Ryzen 9 7950X, host memory 50.6 GB/s at best). Per cell: the deployed cache’s speed; each oracle’s ratio to it with the whole sequence in view (paired, 95% interval), its hit rate, its misses per token (experts read from host memory by the CPU or by a demand copy; its background admissions in parentheses: those of MIN-2R are this step’s CPU-served misses, each read a second time by the copy, those of Belady-2R are prefetches of what MIN would bypass as readily as of what it would admit) and the calibrated model on its own counters against its measured time; the optimum’s reads (R^*) and hit rate from the trace; and MIN-2R simulated on the trace with the engine’s lookahead (within the sequence) under a copy latency of 1 (ideal), 2 and 3 steps, with the latency at which the simulation meets the engine’s misses.

Cell	deployed	ratio	MIN-2R, $W=all$				Belady-2R, $W=all$				optimum R^* hits	simulated misses/token			
			hits	misses (adm.)	model	ratio	hits	misses (adm.)	model			$d=1$	2	3	fit
gpt-oss 11%	48.0	1.03 [1.03, 1.04]	65	50 (16)	+7%	0.97 [0.96, 0.98]	71	42 (39)	+19%	38.3	73	39	43	47	3.9
gpt-oss 25%	78.6	1.18 [1.18, 1.19]	86	20 (9)	+6%	1.63 [1.61, 1.66]	95	7 (23)	+53%	15.3	89	16	17	19	4.7
gpt-oss 40%	113.8	1.23 [1.22, 1.24]	94	9 (4)	+2%	1.56 [1.54, 1.59]	98	3 (12)	+39%	6.8	95	7	8	9	3.5
Qwen3 12.5%	27.3	1.00 [1.00, 1.01]	64	138 (30)	+5%	0.95 [0.94, 0.95]	67	128 (65)	+11%	96.5	75	97	122	143	2.8
Qwen3 25%	45.0	1.13 [1.12, 1.14]	83	64 (21)	+9%	1.41 [1.39, 1.43]	91	34 (52)	+37%	43.4	89	44	54	61	3.4
Qwen3 43.75%	83.2	1.19 [1.18, 1.20]	95	19 (9)	+1%	1.62 [1.58, 1.65]	99	3 (27)	+47%	13.7	96	14	17	19	3.0

layer’s CPU misses with its GPU hits changes speed by under 1% at every budget, and predicting the next layer’s experts (Hwang et al., 2024; Zhu et al., 2025) never lowers host bytes.

F THE TRACE STUDY IN DETAIL

Policies. Table 16 scores LRU, LFU, decayed frequency, ARC, S3-FIFO and a profiled static cache against MIN and against Belady within a window of W future tokens, at three budgets per model, on each model’s own sampled text (9 traces, the cache carried across conversations). Without admitting every miss, the online policies are close to one another and far from MIN: the best reads 35–110% more, decayed frequency is best at 12 of 27 budget points and S3-FIFO at 10. Where the cache turns over every token ($C/k \leq 1.5$) the six are within 4% of one another; from $C/k=2$ LRU reads 2–11% more than decayed frequency on eight of nine models, and classic LFU fails at large budgets. Adding *admit every miss* (every miss copied into the slot of the lowest-scored resident not requested now) changes the ranking: it is best at 18 of 27 points, ties at 5, and loses by at most 2.7%; it saves up to 5.8% over the previous best, and the best online policy still reads 35–110% more than MIN (prereg/policy_aa.json). At the small budgets almost every resident turns over within a few tokens, and the expert just requested is a good bet. In our engine admitting every miss reads all of its misses over the link, so its value in time depends on the link (Section 5). A static cache profiled on other text reads $1.4\text{--}13.2\times$ MIN because the hot set moves: the C most requested experts of a layer change by 22–58% between windows of 100 tokens and 33–75% between windows of 1,000.

The horizon, before and after the stronger baseline. Against decayed frequency with each admission fetched once (the study’s original baseline), MIN reads 26–56% fewer experts (median 41%), and the window that closes half of the gap fitted $W_{50} \approx 0.59 (C/k)^{1.33}$; admitting every miss lowers the baseline at 23 of 26 points by up to 6.7%, raises W_{50} by a median 9% (at most 42%), and refits it to $0.67 (C/k)^{1.31}$, with a leave-one-model-out exponent range of 1.25–1.36 and, without the points below one token, 1.35 (it was 1.51 against the original baseline). Ninety percent of the gap needs a median of $2.7 C/k$ tokens. Table 17 compares the rule in distinct experts with the power law in tokens and with $W_{50} \propto C/k$, each

Table 16. Host reads per token relative to the exact optimum (Belady’s MIN with bypass over the whole future), median over nine models at three budgets, and the range over all 27 model–budget cells. Event-atomic replay of each model’s own sampled text; every online policy is scored on the last 90% of the trace. Mixtral (8 experts) uses $C = 2, 3$ and 4.

Policy	$C=E/8$	$E/4$	$3E/8$	range
LRU	1.44	1.77	2.03	1.37–2.26
LFU	1.44	1.93	2.40	1.37–6.15
Decayed frequency	1.45	1.70	1.94	1.35–2.10
with hysteresis κ	1.46	1.75	2.00	1.36–2.55
ARC	1.44	1.76	1.99	1.37–2.23
S3-FIFO	1.44	1.69	1.94	1.38–2.11
Static, profiled	1.92	2.74	3.48	1.43–13.24
Belady, $W=1$	1.18	1.45	1.69	1.07–2.06
Belady, $W=4$	1.01	1.13	1.31	1.00–1.95
Belady, $W=16$	1.00	1.00	1.01	1.00–1.66

Table 17. Predicting a held-out model’s horizon W_{50} (fitted on the other eight models, tested on the ninth, over all 26 budget points): error as a factor, $\max(\hat{W}/W, W/\hat{W})$.

Rule	Constants	Median	90th pct.	Max
$W_{50} \propto C/k$	1	1.22	1.57	2.42
$W_{50} = a(C/k)^b$	2	1.19	1.51	1.67
$D(W_{50}) = cC$	1, and $D(W)$	1.16	1.42	1.61

fitted on eight models and tested on the ninth: the first two are not distinguished, and both beat the third. The rule in distinct experts inverts the held-out model’s own curve of distinct experts against window length, which needs that model’s routing trace. Across the 9 models, the window that closes half of the gap holds a median 0.65 C distinct experts per layer (quartiles 0.61–0.72).

A learned admission order. We trained a logistic model of whether an expert is requested within the next C/k tokens on features the engine has when it plans a step (decayed request counts at five half-lives, the time since the last request, whether the expert is requested now, and transition scores from the previous token’s experts at the same layer and, through an 8-bit table, at every layer), fitted on each model’s mixed-domain trace and replayed on the AIME routing, so no test text is seen in training (Table 18). With each admission read once it reads 6–12% fewer experts than decayed frequency with the same single-read admission at the four host-bound budgets, closing 17–30% of that policy’s excess over MIN; admitting every miss alone closes most of that at Qwen3 12.5% and little at gpt-oss (prereg/learned_offline.json and the research checks). In the engine (job 097, predictions committed before launch) it reads 6–13% fewer experts than Dep-1R at the host-bound budgets on both machines, but runs 1.01–1.03 $\times$ it on O4’s machine and 0.90–0.94 $\times$ on host O6: the reads it saves are CPU reads, bought with more fetches in the step, each a copy its layer waits on, and with 237–585 μ s of host time per token (over all six budgets).

Table 18. A learned admission order, offline: host reads per token on the AIME-25 routing the engine runs, of the decayed frequency with each admission read once (Dep-1R; within 10% of the engine’s reads on hosts O4 and O5), the learned order with each admission read once (trained on the model’s mixed-domain trace; 8-bit cross-layer table, as in the engine), Belady within one and four tokens of perfect foresight, and MIN with bypass in its no-evict form (Table 28). *closed*: the share of Dep-1R’s excess over MIN that the learned order removes. [†]host-bound.

cell	Dep-1R	learned	$W=1$	$W=4$	MIN	closed
gpt-oss 11% [†]	59.4	56.0	54.4	47.3	39.0	17%
gpt-oss 25% [†]	28.3	26.1	27.3	25.1	15.5	17%
gpt-oss 40%	14.3	12.6	14.0	13.2	6.8	23%
Qwen3 12.5% [†]	166.2	146.3	134.1	109.1	100.4	30%
Qwen3 25% [†]	84.2	74.8	74.1	62.8	44.3	24%
Qwen3 43.75%	28.8	26.3	27.7	25.3	13.8	17%

Table 19. Partial foresight on the AIME routing the engine runs: host reads per token of admitting every miss and of MIN-1R with the whole problem in view (R_{eng}^* , Appendix E), and the share of the gap between them that each window closes (Table 2). Recall 0.5: each future expert kept with probability 0.5 and replaced by a random one (fill) or left out (drop); means of three seeds. Linear: the ridge forecaster of Appendix G with a 4-token window. $\kappa=1$: the same exact 4-token window with the deployed margin instead of admitting every miss beyond it, as a share of its own gap.

Budget	Reads per token		Exact window W (tokens)					Recall 0.5, fill / drop		Linear	$\kappa=1$
	every miss	R_{eng}^*	1	2	4	8	16	$W=8$	all	$W=4$	$W=4$
gpt-oss 11%	58.6	39.3	0.18	0.31	0.52	0.81	0.98	0.29 / 0.29	0.43 / 0.48	0.13	0.62
gpt-oss 25%	28.2	16.0	0.07	0.14	0.25	0.42	0.66	0.14 / 0.13	0.46 / 0.47	0.05	0.34
Qwen3 12.5%	150.0	100.3	0.31	0.51	0.77	0.96	0.99	0.42 / 0.43	0.48 / 0.52	0.19	0.88
Qwen3 25%	79.6	44.9	0.16	0.28	0.46	0.70	0.93	0.29 / 0.28	0.49 / 0.53	0.11	0.60

Speculation is not foresight. Speculative decoding verifies K draft tokens in one forward pass, so a layer’s experts for K positions are requested together and a missed expert is read once for all of them. On the same traces, with drafts accepted at rate α until the first rejection, batched verification saves an online cache a median 15–32% of its reads at $K=8$ when rejected drafts cost nothing, the equivalent of a window of two to four tokens; once the rejected drafts’ experts are counted, the saving is 8–12% at $\alpha=0.9$ and becomes a loss of 18–20% at $\alpha=0.8$, because the union of K positions’ expert sets is 20–79% of Kk . MIN itself gains little from batching (–7 to 15%). Systems that use drafts as a look-ahead sensor (Li et al., 2026) compare against a baseline that already pays for verification; ours charges the rejected drafts to a cache that does not speculate.

G PARTIAL FORESIGHT IN DETAIL

The window policies. At step g the engine plans the step’s misses before it runs, from the trace’s record of step g (the routers give it layer by layer in a deployable system) and from a view of records $g+1..g+W$ of the same problem. A miss is processed in request order: it takes a free slot; otherwise the victim is a resident not requested now that is not seen in the window (lowest decayed score), or, if every resident is seen, the one seen furthest; a miss seen in the window is admitted unless the victim is seen sooner, and a miss not seen is admitted if the victim is not seen either (admit every miss beyond the window; with the deployed margin $\kappa=1$, only if its score beats the victim’s by one). On the panel admissions are fetched in the step, so every miss is read once; on the deployed path of the follow-up the machine’s fetch table splits the misses between the CPU and an in-step copy, and an admission served by the CPU is copied afterwards, a second read. In a degraded view each expert of a future record is kept with probability r , decided by a hash of (seed, g , record, layer, position), so step g sees one view throughout and step $g+1$ an independent one; a dropped expert is replaced by a uniformly drawn one (fill: precision and recall r) or left out (drop: recall r , precision 1). `scripts/value_map.py` replays the plan bit for bit: on a CPU build the engine’s reads equal the replay’s in 30 configurations at three budgets (`cputest099`), and on the panel’s GPUs, where the routing in the run can differ from the recorded trace by numerics, within 1.7% at both gpt-oss budgets. The panel ran the first 20 of the 30 problems; the table’s reads are on all 30. Table 20 lists the windows on the deployed path and in the step on the machines of job 100. Copied in the step, a window beats the deployed path where the link reads at least 0.92 of the CPU rate and loses to it at 0.54 or below; on the deployed path the engine misses 7–33% more than an instant replay (the deployed state matches its replay within 2.4%), and background copies that land late are the likely cause.

Table 20. Foresight on each read path (job 100): speed relative to the deployed cache on the same machine. *In step*: admit every miss with a window of W exact tokens, each admission copied in the step (as on the panel). *Deployed path*: the deployed policy with the same window, its misses split between the CPU and an in-step copy by the machine’s fetch table and its admissions copied in the background; $W=8$ at recall 0.5 in the column marked r . The last three columns: MIN-2R, MIN-1R and the read-ahead oracle. Machines sorted by the probe’s link-to-CPU ratio.

Machine	Link/ CPU Budget		In step $W=4$ 16		Deployed path 4 16 r			Read- ahead		
Pd again	0.41	gpt-oss 11%	0.66	0.89	1.02	1.03	0.94	1.00	1.05	1.11
Pd again	0.41	gpt-oss 25%	0.69	0.81	1.03	1.12	1.01	1.19	1.11	1.29
9950X, slower link	0.54	gpt-oss 11%	0.65	0.89	1.01	1.01	0.96	0.99	1.07	1.23
9950X, slower link	0.54	gpt-oss 25%	0.69	0.80	0.96	1.06	1.00	1.10	1.11	1.37
5700X3D	0.92	gpt-oss 11%	1.09	1.32	0.99	1.00	0.95	0.99	1.37	1.44
5700X3D	0.92	gpt-oss 25%	1.06	1.23	1.03	1.07	1.00	1.16	1.48	1.80
Ph again	0.96	gpt-oss 11%	0.99	1.20	1.01	1.03	0.98	1.03	1.27	1.45
Ph again	0.96	gpt-oss 25%	0.97	1.09	1.03	1.07	1.01	1.16	1.29	1.64

What the table says. Horizon and accuracy trade: at gpt-oss 11% an 8-token window at recall 0.5 closes 0.29 of the gap between admitting every miss and R_{eng}^* , about what two exact tokens close (0.31), and the whole future at recall 0.5 closes 0.43. Fill and drop cost about the same for windows of up to 16 tokens; with the whole future in view at low recall, dropping costs less (the largest difference, 0.12, is there). A larger cache needs a longer window: at 25% the same exact 4-token window closes 0.25 of the gap, against 0.52 at 11%.

A realisable forecaster. Ridge regression from the all-layer routing of step t to that of step $t+h$ ($h=1..8$), fitted on each model’s mixed-domain own text and applied to the AIME routing, forecasts the top k experts of each layer with a precision of 0.48 one token ahead and 0.26 four ahead on gpt-oss (0.54 and 0.32 on Qwen3); a decayed request count, the deployed cache’s own signal, scores 0.30 and 0.28. Driving the window policy, the ridge forecasts close 3–15% of the gap on gpt-oss and 6–19% on Qwen3 over windows of 1–8 tokens (`prereg/forecaster_linear.json`). Its precision falls with the horizon, so over a window it is closer to 0.3 than to its one-token value; at that average precision a window with random errors closes about as much on gpt-oss, and more on Qwen3, where the forecaster’s correct forecasts are mostly experts the cache already holds. A sequence model reports recall above 90% one token ahead with $k+3$ candidates (Wang et al., 2026b); its precision is then at most $k/(k+3)$, 0.57 for gpt-oss and 0.73 for Qwen3. A recurrent forecaster in that spirit (a GRU of 512 units over the same all-layer vectors, one output head per horizon, trained on the same text; `scripts/forecaster_gru.py`) reaches a precision of 0.40 and 0.26 one and four tokens ahead on gpt-oss (0.46 and 0.31 on Qwen3), a recall at $k+3$ of 0.53 and 0.54, and closes 2–11% and 6–17% of the gap: no better than the linear model. Our training text is smaller than SeqMoE’s and differs from the test text, so this bounds what such a model does across text, not what it can do in domain.

H A CLOSED-FORM ACCOUNT OF THE TIME, AND ITS REGISTERED TESTS

This appendix holds the account of the deployed cache’s time that the main text summarises in one paragraph (Section 4): what it is, how it was found, how it fared in the tests registered on machines new to it, and the split of the time it implies. The measured decomposition of Section 4 does not rely on it.

H.1 The relation

For the deployed cache, we describe the time per token T by

$$T = G + \frac{(M + (1 - G/T) A) S}{B_{\text{host}}}, \quad (4)$$

solved for T : it is the larger root of $T^2 - (G + t_M + t_A) T + t_A G = 0$, with $t_M = MS/B_{\text{host}}$ and $t_A = AS/B_{\text{host}}$. M and A are the misses and the background admissions per token (the engine’s counters), S the expert’s bytes, B_{host} the

Table 21. The four registered tests of Eq. (4). A *cell* is one model at one budget on one machine. *New*: machines no earlier launch had rented. Each test registered a band for every cell and a median error of at most 4%; job 108 also allowed at most one cell beyond 6%. Jobs 107 and 108 count only the machines that passed their registered validity checks; job 107 registered that with fewer than three it is inconclusive. *New machines*': their cells within the band. The last column sets aside, after the fact, the hosts the text discusses: in job 106 the two that computed wrong outputs or ran unsteadily, in job 108 the one whose GPU compute could not be profiled.

Job	Form	Models	Band	Machines (new)	Cells within	New machines'	Median	As registered	Set aside
105	plain	gpt-oss	6%	4 (2)	5/8	2/4	4.2%	failed	–
106	overlap	both	6%	5 (2)	12/20	0/8	4.7%	failed	held (12/12, 2.3%)
107	overlap	both	6%	2 (2)	1/8	1/8	31.7%	inconclusive	–
108	overlap	gpt-oss	8%	3 (3)	4/6	4/6	6.7%	failed	failed (4/4, 5.3%)

machine's best probed rate, and G the GPU's own kernel time per token in an Nsight profile of the same engine (4.1–4.7 ms for gpt-oss; where a machine was not profiled or its profile captured no decode kernels, the median of a fixed set of 7 profiles; Appendix I). G exceeds T_{GPU} of Eq. (2) because it also counts the expert kernels the GPU runs for resident and fetched experts. No constant is fitted to the times. Every host read costs host time, except the background copies that run while the GPU computes: they spread over the next step, and a share G/T of them falls under the GPU's compute. The *plain* form drops that share. We found the plain form on 30 launches of gpt-oss (exploratory): it is within 6% on 28 of them at 11% but over-predicts at 25%, where admissions are a larger share of the reads. The overlap term came from where those copies run, after the plain form failed its first registered test; over 34 launches it brings 33 instead of 18 within 6% at 25%, at the cost of 29 instead of 32 at 11%.

H.2 Registered tests

After the plain form failed, we registered Eq. (4) before three more experiments, each predicting it within a band at every cell, with G profiled on each machine before its timed runs (Table 21). None passed as registered.

The first test: held on machines rented before, failed on new ones. On new runs of 3 machines we had rented before, it came within 6% at all 12 cells of both models (median error 2.3%). This does not single out the overlap term: discounting admissions by half passes 11 of those cells. The same test failed on the two server machines new to it, by 10–52%; one computed wrong outputs and the other ran its rounds 52% apart (slowest over fastest), so the following tests set validity gates before launch (Appendix I).

Machines never rented before: not passed. The next test admitted only machines whose GPU no earlier launch had recorded, with little host memory in use, correct outputs and rounds within 2% of each other. Only two of 7 passed, short of the three required, so by its own rule the test is inconclusive; the relation missed the band at 7 of their 8 cells, by up to 59% on a dual-socket server (51% against the probe's second-highest reading) and 11% on a Ryzen 9 5900XT. A last test restricted the machines in advance to the class the relation had been found on (one NUMA node, at most 32 usable cores) and widened the band to 8%. It failed on an EPYC 7543 inside that class, by 34.3%, and without that machine still failed two pooled clauses (Appendix I).

The error depends on the budget. On the three consumer machines new to the last two tests the relation under-predicted every gpt-oss cell. Over all 25 consumer machines its mean error is -2.2% at gpt-oss 11% (95% t -interval over machines -4.0 to -0.4) and $+1.6\%$ at 25% (0.0 to 3.1): what Eq. (4) leaves out changes sign with the budget.

H.3 After the fact: the processor

The processor separates most of the machines the relation fits from those it misses, not all; we drew this split after the tests. On 39 valid launches on 25 machines with consumer processors, the read rate the deployed cache implies, $(M + (1 - G/T)A)S/(T - G)$, is 0.88–1.16 of the probe's best rate at gpt-oss 11%, and the relation is within 8% on 37 of them (G was profiled on the machine for 9 of these launches). On server processors, 5 of 8 launches failed a validity check, and the three valid ones read at AMD engineering sample 0.27, EPYC 7543 0.58, EPYC 7402P 1.04 of it: the last fits the relation, the other two do not. We have not established why. On the dual-socket server the probe's own threads read slower with 128 threads, about the engine's 126 helpers, than with 16 (118 against 151 GB/s), and on an EPYC 9655 rented earlier the per-layer timers showed a latency floor no consumer machine has (the supplement).

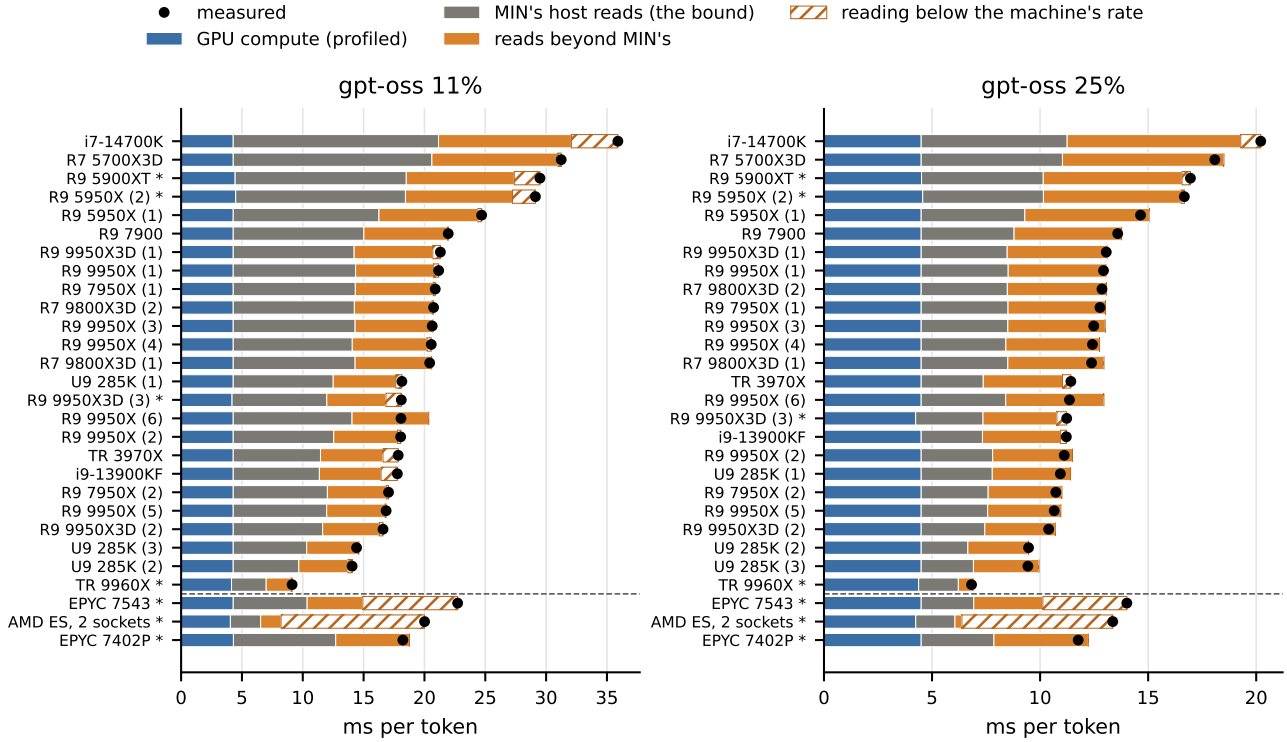


Figure 7. The split implied by Eq. (4) (exploratory): the deployed cache’s time per token at the two host-bound gpt-oss budgets, machine by machine, split into the GPU’s compute (G , profiled on the machine, or the median profile where none was taken or it captured no decode kernels), MIN’s host reads at the machine’s best rate (the bound of Eq. (1)) and the reads the cache makes beyond MIN’s. The dot is the measured time; hatched, the measured time beyond Eq. (4), reading below the machine’s best rate; a bar that ends past the dot is an over-prediction. Machines with the same processor are numbered in order of rental. One launch per machine that ran stably: 25 with consumer processors (above the dashed line) and 3 with server processors (below). *: first rented for a registered test of Eq. (4).

H.4 The split it implies (exploratory)

On consumer processors, then, the relation splits the deployed cache’s time into three parts (Fig. 7). At gpt-oss 11% the GPU’s compute takes 12–45% of the time, MIN’s reads, which are the bound, 31–54%, and the reads beyond MIN’s 22–35%. The first is serialisation, the GPU’s compute in series with the reads, and makes up 23–66% of the gap to Eq. (1); the second is the reads MIN would not make. With G removed, the time scales with the machine’s read rate with an elasticity of -0.95 (95% interval -1.06 to -0.84 over the machines of the exploratory set), as reads at B_{host} would. Foresight can attack both parts: fewer reads need MIN’s choices, and overlap needs reads issued ahead. Reading below the probed rate takes at most 11% of the time on the consumer machines; on the server machines it is a third part (EPYC 7402P 0%; EPYC 7543 34%; AMD ES, 2 sockets 59% of the time at 11%).

I MIN’S SCHEDULES AND THE MACHINE, IN DETAIL

The fewest-admission schedule, machine by machine. Tables 22 and 23 list jobs 104 and 105: MIN’s set loaded by the CPU (MIN-2R, Few-2R) or in the step (MIN-1R, Few-1R), as the greedy rule picks it or as the schedule with the fewest admissions does. On the engine’s 20 problems the fewest-admission schedule copies 0.60–0.73 times as many experts in the step as the greedy one, for 2.5–5.3% more misses (more than its replay predicts; its measured gains include that cost). On every launch it gains over the deployed cache with either load and at both budgets, each 95% interval above 1; on the machine with the fastest link, Few-2R is 0.07 behind MIN-2R at 25%.

Table 22. Which of MIN’s hit-optimal sets the oracle follows (job 104): speed relative to the deployed cache on the same machine, with MIN’s set loaded by the CPU (two reads: MIN-2R, Few-2R) or in the step (one read: MIN-1R, Few-1R), as the greedy rule picks it or as the schedule with the fewest admissions does; and the copies per token each makes in the step. Machines sorted by the probe’s link-to-CPU ratio.

Machine	Link/ CPU Budget		By the CPU		In the step		Copies/token	
			MIN-2R	Few-2R	MIN-1R	Few-1R	MIN-1R	Few-1R
Pf again	0.29	gpt-oss 11%	0.94	1.11	0.86	1.04	19.8	11.8
Pf again	0.29	gpt-oss 25%	1.13	1.22	0.96	1.06	9.7	7.0
285K, second	0.49	gpt-oss 11%	1.03	1.08	1.05	1.18	19.8	11.8
285K, second	0.49	gpt-oss 25%	1.20	1.21	1.12	1.17	9.7	7.0
Pg again (5950X)	0.63	gpt-oss 11%	1.02	1.06	1.26	1.34	19.8	11.8
Pg again (5950X)	0.63	gpt-oss 25%	1.20	1.15	1.32	1.35	9.7	7.0

Table 23. Job 105, registered before the runs, nothing fitted: the deployed cache’s time per token against the plain form of Eq. (4) (without the overlap term, which came after this job), the GPU’s compute profiled on the same machine before the timed runs plus the run’s counted host reads at the machine’s best rate; and the speed, relative to the deployed cache, of LA, the deployed cache with the engine’s layer-ahead copy (each layer’s top predicted expert of the next layer copied ahead of use), and of MIN-1R and Few-1R, MIN’s greedy and fewest-admission sets copied in the step. Machines sorted by the probe’s link-to-CPU ratio.

Machine	Link/ CPU Budget		G_{prof} (ms)	Deployed (ms)		LA	MIN-1R	Few-1R
				plain Eq. 4	measured			
Pf again	0.28	gpt-oss 11%	4.1	13.6	13.9	0.72	0.86	1.03
Pf again	0.28	gpt-oss 25%	4.4	9.8	9.5	0.69	0.97	1.06
Threadripper 9960X	0.32	gpt-oss 11%	4.1	9.3	9.1	0.91	0.91	1.04
Threadripper 9960X	0.32	gpt-oss 25%	4.4	7.3	6.8	0.87	0.98	1.04
EPYC 7402P	0.45	gpt-oss 11%	4.3	19.1	18.2	0.81	1.01	1.15
EPYC 7402P	0.45	gpt-oss 25%	4.5	12.9	11.8	0.78	1.09	1.15
O4 again (9950X)	1.04	gpt-oss 11%	4.4	20.8	20.5	1.04	1.36	1.34
O4 again (9950X)	1.04	gpt-oss 25%	4.4	13.2	12.4	1.01	1.42	1.39

Job 106, every machine. Table 24 adds the two server machines new to job 106. On the EPYC 7302 machine the engine computed wrong outputs: its teacher-forced loss on the deployed cache was 0.21–0.45 nats per token across rounds and models, against 0.19 (gpt-oss) and 0.085 (Qwen3) on every other machine and in its own untimed routing pass, and its counters changed between rounds. On the Xeon 8347C machine the deployed cache ran its gpt-oss 11% rounds 52% apart (slowest over fastest), against at most 2% registered, and the greedy copy gained in two of its three rounds where we had predicted it would lose. When their jobs reached the gate, before any download, free reported 131–132 GB of host memory in use (12–15 GB on the other machines), and their probes’ CPU read rates fell when more threads read than the container had usable physical cores. On both, Eq. (4) under-predicted the deployed cache by 10–52%. We report their clauses (in the supplement) and draw nothing from them about the relation on those runs; both are servers, and job 107’s one valid server was under-predicted as well (Appendix H). The three machines that ran stably were all rented before: Pf, the Threadripper 9960X of job 105, and panel host Pe (same GPU).

Job 107, every machine. Table 25 lists the four machines that passed the gate before any download. The EPYC 9754 and the EPYC 7663 ran their gpt-oss 11% rounds 3.5% and 6.0% apart, beyond the registered 2%, and stopped there, as registered; the relation under-predicted them by 31% and 21%. The others stopped at the gate with 50–96 GB of host memory in use. The registration replaced only machines that failed the gate, from a list of offers in order; replacing machines that failed the round check was added during the job, after the results of the first four machines were in, in an amendment committed before the machine it added started. By then one listed offer was gone and the next would have exceeded the

Table 24. Job 106, registered before the runs, every machine (*: deployed cache varied by more than the registered 2% between rounds): the deployed cache’s time per token against Eq. (4) (GPU compute profiled on the same machine before the timed runs; the run’s own counters), and the speed relative to the deployed cache of the two online admission rules (*Margin*: the decayed count with a larger admission margin; *lrn*: the learned reuse predictor with a margin; both chosen on other text) and of **Few-1R**, MIN’s fewest-admission set copied in the step, with round-level 95% intervals (rounds, then problems: three rounds at gpt-oss 11%, two at 25%; Qwen3 ran one round, so its intervals are over problems). Machines sorted by the probe’s link-to-CPU ratio.

Machine	Link/ CPU Budget		G Deployed (ms)					lrn		Few-1R	
			(ms)	Eq. 4	meas.	Margin					
EPYC 7302*	0.14	gpt-oss 11%	4.3	12.2	17.7	1.06	[1.01, 1.10]	0.97	[0.92, 1.01]	0.85	[0.82, 0.88]
EPYC 7302*	0.14	gpt-oss 25%	4.7	8.7	12.2	1.08	[1.00, 1.17]	1.10	[1.06, 1.14]	–	
EPYC 7302*	0.14	Qwen3 12.5%	5.8	20.4	25.4	1.07	[1.02, 1.13]	0.93	[0.86, 1.00]	–	
EPYC 7302*	0.14	Qwen3 25%	6.3	14.3	17.2	1.01	[0.93, 1.08]	0.84	[0.76, 0.94]	–	
Xeon 8347C*	0.23	gpt-oss 11%	4.1	12.0	19.5	1.28	[0.99, 1.56]	1.08	[0.80, 1.45]	1.33	[1.00, 1.61]
Xeon 8347C*	0.23	gpt-oss 25%	4.5	8.6	10.1	1.06	[1.05, 1.08]	1.02	[0.99, 1.04]	–	
Xeon 8347C*	0.23	Qwen3 12.5%	5.3	21.4	45.0	1.96	[1.67, 2.24]	1.29	[0.94, 1.74]	–	
Xeon 8347C*	0.23	Qwen3 25%	5.8	14.9	16.6	0.86	[0.71, 1.02]	0.73	[0.59, 0.88]	–	
Pf again (285K)	0.29	gpt-oss 11%	4.3	13.8	13.7	1.02	[1.01, 1.04]	1.03	[1.02, 1.03]	1.02	[1.01, 1.03]
Pf again (285K)	0.29	gpt-oss 25%	4.7	9.6	9.5	1.06	[1.04, 1.07]	1.05	[1.04, 1.07]	–	
Pf again (285K)	0.29	Qwen3 12.5%	5.4	24.8	24.4	1.02	[1.00, 1.03]	1.02	[1.01, 1.03]	–	
Pf again (285K)	0.29	Qwen3 25%	5.9	16.9	16.2	1.04	[1.03, 1.04]	1.03	[1.02, 1.03]	–	
Threadripper 9960X again	0.32	gpt-oss 11%	4.1	9.1	9.4	1.00	[0.99, 1.02]	0.98	[0.97, 1.00]	1.06	[1.05, 1.06]
Threadripper 9960X again	0.32	gpt-oss 25%	4.4	6.9	7.0	1.02	[1.00, 1.04]	0.99	[0.97, 1.00]	–	
Threadripper 9960X again	0.32	Qwen3 12.5%	5.4	15.1	14.4	1.02	[1.01, 1.02]	0.99	[0.99, 1.00]	–	
Threadripper 9960X again	0.32	Qwen3 25%	5.7	11.2	10.5	1.01	[1.01, 1.01]	0.98	[0.98, 0.99]	–	
Pe again (9950X)	1.02	gpt-oss 11%	4.5	20.4	20.5	1.01	[1.01, 1.01]	1.01	[1.01, 1.02]	1.34	[1.33, 1.35]
Pe again (9950X)	1.02	gpt-oss 25%	4.5	12.8	12.4	1.01	[1.01, 1.01]	1.01	[1.00, 1.02]	–	
Pe again (9950X)	1.02	Qwen3 12.5%	5.8	35.0	36.7	1.01	[1.01, 1.01]	1.02	[1.02, 1.02]	–	
Pe again (9950X)	1.02	Qwen3 25%	5.9	22.2	22.5	1.01	[1.01, 1.01]	1.01	[1.01, 1.02]	–	

remaining credit, so the last two machines were chosen by rules committed before each started (the cheapest verified offer not rented before with cheap traffic and downloads of at least 1000 and 900 Mb/s), and both stopped at the gate. On the two valid machines, the relation predicted the admission margin’s speed from its own counters within a median 0.016 (up to 0.067, on the server), against 0.021 for predicting no change. The profiled GPU compute was 4.1–4.5 ms on gpt-oss and 5.6–5.9 ms on Qwen3, within the range of earlier machines.

Job 108, every machine. Table 26 lists the machines that ran. The EPYC 7K62 ran its two gpt-oss 11% rounds 7.0% apart and stopped, as registered; the relation under-predicted it by 26.8%. The gate stopped a Ryzen 7 5700X3D whose GPU job 100 had rented, and an EPYC 9354 machine with 323 GB of memory in use. Machines were rented from the registered list in order until three were valid. On both EPYC machines the Nsight trace held almost no kernels (driver 570; the Ryzen machines ran drivers 580 and later), so their *G* is the median of the 7 profiles of jobs 069c and 105, as everywhere else (4.28 and 4.50 ms). This is a deviation from the registration that does not change the outcome: without the EPYC 7543 the median error over the 4 cells is 5.3% and all are within 8%, but two pooled clauses still fail. The registered offer list said none of its offers was in the rental ledger, but three were: the 5700X3D’s (job 100; the gate turned it away), the 9950X3D’s (rented for a few minutes in job 099 without producing data; its GPU matched no earlier launch’s) and an i9-13900KF’s that was never reached.

The probe’s rate, and machines as the unit. B_{host} is the highest single reading of one probe run. Its second-highest reading is a median 0.8% lower and more than 5% lower on 1 of 42 launches (28 machines): launch 107d (Table 25), by 26% (Table 27). With the second-highest reading, Eq. (4) is within 6% on 34 instead of 31 of the 39 consumer launches at gpt-oss 11%; with the best CPU-only reading, which ignores that the link reads at the same time, on 22. By Eq. (4)’s own accounting the engine reads more than 5% faster than the highest reading on 12 of 84 launch-budgets (up to 1.22 $\times$), so the bounds are relative to the probe. Taking the machine as the unit, one value per machine: **Few-1R** gains a mean 1.24 $\times$ (95% *t*-interval over machines 1.16–1.32) on the 13 machines that ran stably; **Margin**, each machine’s mean over the two gpt-oss budgets, 1.020 $\times$ (1.004–1.035) on 5. Equation (4)’s error is a mean -2.2% (-4.0 to -0.4%) at gpt-oss 11% over the 25

Table 25. Job 107, registered before the runs, on the machines never rented before that passed the registered gate before any download ($\dagger$: rounds further apart than the registered 2%; such a machine stopped after its gpt-oss 11% rounds and is outside the predictions): the deployed cache’s time per token against Eq. (4) (GPU compute profiled on the same machine before the timed runs; the run’s own counters), and the speed relative to the deployed cache of Margin and of MIN’s fewest-admission set copied in the step or loaded by the CPU, with round-level 95% intervals (rounds, then problems: two rounds of gpt-oss, one of Qwen3, whose intervals are over problems). Machines sorted by the probe’s link-to-CPU ratio.

Machine	Link/ CPU Budget		G Deployed (ms)				Few-1R, Few-2R			
			(ms)	Eq. 4	meas.	Margin	in the step		by the CPU	
EPYC 7663 $\dagger$	0.21	gpt-oss 11%	4.3	11.3	14.3	1.03 [1.00, 1.05]	0.97 [0.94, 1.01]	1.13 [1.07, 1.18]		
AMD engineering sample	0.44	gpt-oss 11%	4.1	8.2	20.0	1.05 [1.04, 1.06]	1.16 [1.14, 1.17]	1.13 [1.11, 1.14]		
AMD engineering sample	0.44	gpt-oss 25%	4.2	6.4	13.4	0.99 [0.98, 1.00]	–	–		
AMD engineering sample	0.44	Qwen3 12.5%	5.6	13.6	32.7	1.03 [1.02, 1.04]	–	–		
AMD engineering sample	0.44	Qwen3 25%	5.9	10.4	23.7	1.07 [1.06, 1.09]	–	–		
EPYC 9754 $\dagger$	0.51	gpt-oss 11%	4.2	10.1	14.7	0.97 [0.95, 0.99]	1.18 [1.15, 1.20]	1.05 [1.02, 1.08]		
Ryzen 9 5900XT	0.75	gpt-oss 11%	4.4	27.4	29.5	1.02 [1.02, 1.02]	1.42 [1.41, 1.43]	1.04 [1.04, 1.04]		
Ryzen 9 5900XT	0.75	gpt-oss 25%	4.5	16.6	17.0	1.02 [1.01, 1.02]	–	–		
Ryzen 9 5900XT	0.75	Qwen3 12.5%	5.8	48.3	54.4	1.02 [1.02, 1.02]	–	–		
Ryzen 9 5900XT	0.75	Qwen3 25%	5.9	29.7	31.7	1.02 [1.02, 1.02]	–	–		

Table 26. Job 108, registered before the runs and restricted in advance to desktop-class machines never rented before: the deployed cache’s time per token (gpt-oss, mean of two rounds) against Eq. (4), with G profiled on the same machine before the timed runs (on the two EPYC hosts, whose traces held no decode kernels, the median of the profiles of jobs 069c and 105), the run’s own counters and the probe’s best rate, and the read rate the time implies as a share of that rate. $\dagger$: rounds further apart than the registered 2%; outside the predictions.

Machine	Cores	B_{host}	Budget	G (ms)	Eq. (4)	Measured	Implied/best
EPYC 7K62 $\dagger$	23	22	11%	4.3	46.2	63.1	0.71
Ryzen 9 5950X	16	36	11%	4.5	27.2	29.1	0.92
Ryzen 9 5950X	16	36	25%	4.6	16.5	16.7	0.99
Ryzen 9 9950X3D	16	65	11%	4.2	16.8	18.1	0.91
Ryzen 9 9950X3D	16	65	25%	4.2	10.8	11.2	0.94
EPYC 7543	30	84	11%	4.3	14.9	22.7	0.58
EPYC 7543	30	84	25%	4.5	10.1	14.0	0.61

consumer machines, and +1.6% (0.0 to 3.1%) at 25%; on the 21 machines it was found on, -1.5% (-3.5 to 0.4%) at 11%: of the 30 launches it was found on, it under-predicts 25 at 11% and over-predicts 23 at 25%. G was profiled on the machine for 9 of the 39 consumer launches; the others use the median of the profiles. The median |error| is 3.2% on the launches with their own G and 1.7% on the others, but the profiled launches all come from the registered tests.

Admitting less, in detail. The margin κ of Margin was chosen as the value that minimises the deployed path’s reads on each model’s mixed-domain text, not the AIME text (`scripts/online_admit.py`): 3 at the smallest budget and 2 at 25%. It cuts the deployed cache’s host reads by 4–6% at gpt-oss 11% and 6–10% at 25%. On the three machines of job 106 that ran stably it ran $1.00\text{--}1.02\times$ and $1.01\text{--}1.06\times$ the deployed cache (Qwen3: $1.01\text{--}1.04\times$; Table 24). We had registered at least 1.01 at gpt-oss on every machine: it fell short on two of the three stable ones, and the registered pooled median of at least 1.02 held only through the two unstable machines (1.01 on the stable ones). On the two valid machines of job 107 it ran $0.987\text{--}1.074\times$ (median 1.021), below the registered 0.995 at one cell, the server at gpt-oss 25%. Predicted from each configuration’s own counters, Eq. (4) gives these ratios within a median 0.013 on job 106’s stable machines, but predicting no change does almost as well (0.016), and on job 107’s server it missed the registered 0.03 at two cells. A learned reuse predictor with the same kind of margin (job 097’s, fitted on other text) reads less still (7–14% fewer) but costs 267–439 μs of host time per token on gpt-oss and runs $0.98\text{--}1.05\times$: it pays where host memory is slow and loses where it is fast, against the registered prediction that it would not lose.

The slowest link. On the EPYC 7302 machine of job 106, whose link moves 13 GB/s, Few-1R lost in each round ($0.82\text{--}0.88\times$); that machine computed wrong outputs throughout, so we draw nothing from it, but below some link rate even the fewest-admission set should not be copied in the step.

Table 27. How the probe’s rate changes the picture, over the 84 launch-budgets at gpt-oss 11% and 25% of the machines that passed our checks: Eq. (4) with the highest probe reading (as in the paper), the second-highest, and the best CPU-only reading. *Faster than the rate:* launch-budgets whose implied read rate exceeds the rate by more than 5%. The error columns are the consumer launches at 11%.

Rate	Faster than the rate (max)	Within 6%	Within 8%	Median error
highest reading	12 (1.22 $\times$)	31/39	37/39	1.7%
second-highest	15 (1.26 $\times$)	34/39	37/39	1.5%
best CPU-only	46 (1.53 $\times$)	22/39	29/39	5.6%

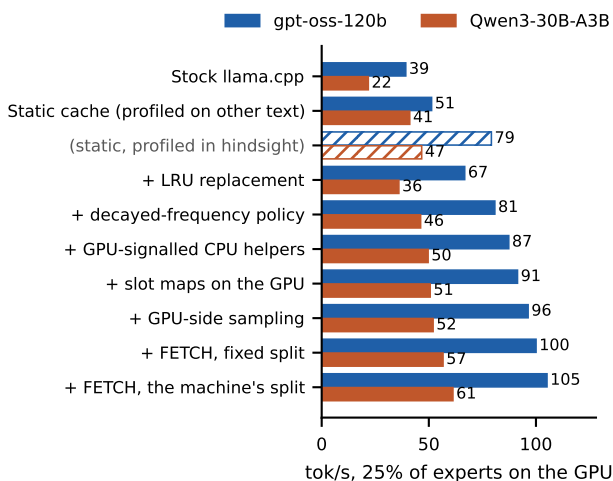


Figure 8. Ablation at 25% of experts on the GPU (15 AIME-25 problems, 256 tokens, host A). Each step adds one component to the one above; the hatched row is not a step: a static cache of each layer’s 32 most requested experts counted on the measured problems themselves.

The machine decides how much. Across machines, the gain of every state that moves reads from the CPU to the link follows the link-to-CPU ratio (Spearman over the machines that ran it through job 104, 95% bootstrap interval): MIN-1R 0.89 [0.63, 0.97] (19 machines), the read-ahead oracle 0.87 [0.54, 0.99] (17), admitting every miss 0.95 [0.75, 0.99] (13). MIN-2R, which reads every admission twice and copies it in the background, tracks the link’s absolute rate more closely (0.79 [0.49, 0.92], 17 machines, against 0.45 for the ratio; the two intervals overlap, and the feature was picked from a search over all probe features). MIN-1R gains 1.02–1.05 $\times$ at 11% on 2 further machines with ratios 0.40 and 0.48, and loses only on the machines whose link reads at under a third of their CPU rate; issued ahead of use, as in the read-ahead oracle, the same copies lose beyond their interval on no machine they ran on (at worst 0.997 $\times$, on Pf).

Machines differ more than problems. Problems rank largely the same on every machine: Kendall’s coefficient of concordance of MIN-1R’s per-problem gain over 19 machines and 20 problems is 0.77 at 11% and 0.82 at 25%, and a problem’s gain follows how far the deployed policy’s reads exceed MIN’s on its trace (Spearman 0.88 at 11%, 0.68 at 25%). Yet machines carry 96% of the variance of its log gain and problems 1–3%.

J THE EXPERT CACHE IN MORE DETAIL

This section holds the measurements of Section 2 that the main text summarises: where the cache’s gain comes from, the choice of the FETCH split, the machines with half the link, long outputs, FreeToken’s own headline model, two other llama.cpp-based systems, output parity, and the cost of prompts and batches. Job numbers refer to the scripts on the gpu branch.

Where the gain comes from. Figure 8 builds the system up from stock llama.cpp on one machine. Placing each layer’s hot experts is the first gain: a static cache of the 32 experts per layer most requested on other text (the models’ own MATH-500

Table 28. The bound under each tightening, tokens per second on host B, and our cache as a percentage of the bound as published (exact optimum, highest probed host rate, datasheet GPU rate) and with every tightening at once. *no-evict*: Belady’s MIN with bypass that keeps a served expert for the step; *exact*: the same, free to evict an expert after serving it; *pool*: one pool of *LC* slots shared across layers; *median*: B_{host} the median of six concurrent probe samples instead of the highest; *GPU*: the GPU term from the measured all-in-VRAM time instead of the datasheet rate; *layers*: the per-layer sum, which no system without cross-layer prefetch can beat; *all*: exact, pool, median and GPU together (a warm start, not included, changes the bound by at most 1.4%).

Model	Experts on GPU	no-evict	exact	pool	median	GPU	layers	all	Ours, % of exact	% of all
gpt-oss-120b	11%	169	172	182	159	172	163	168	41	41
gpt-oss-120b	25%	427	430	474	397	279	365	278	25	39
gpt-oss-120b	40%	519	519	519	517	279	500	278	29	55
Qwen3-30B-A3B	12.5%	92	96	101	89	96	93	93	42	43
Qwen3-30B-A3B	25%	209	214	234	197	192	193	191	30	33
Qwen3-30B-A3B	43.75%	308	308	308	307	192	298	191	35	57

outputs) runs $1.31\times$ and $1.89\times$ stock, which pins whole layers instead. Its hit rate (52% and 67%) depends on how well the profile matches the workload: profiled in hindsight on the measured problems it hits 75% and 72% and runs faster (hatched). Replacing experts online closes that gap without a profile, if admissions are rationed: LRU admits every miss, and on Qwen3 its copies raise host reads from 127 to 188 experts per token, so it runs $0.88\times$ the static cache ($1.30\times$ on gpt-oss); the decayed-frequency policy reads 41 and 118 per token, runs $1.58\times$ and $1.12\times$ the profiled static cache, and the hindsight cache runs 0.98 [0.95, 1.01] and 1.01 [0.98, 1.04] times as fast as it. On traces LRU misses only 2–11% more than decayed frequency at these budgets (Appendix F); the rest of its cost here is the copies it admits. On host S, with everything else as in the full system, LRU runs 6–21% slower than decayed frequency across the six cells of Table 3 and trails FreeToken at four of them: the admission policy is most of the lead over FreeToken (77–84% of it at gpt-oss 25 and 40%, more than the whole of it at gpt-oss 11% and Qwen3 12.5 and 25%). The GPU-signalled CPU helpers, GPU slot maps, GPU sampling and FETCH add 2–9% each, and the model’s FETCH table 5–8% over the fixed one. The full system runs $2.68\times$ (gpt-oss) and $2.81\times$ (Qwen3) stock llama.cpp.

How tight is the bound? Table 28 changes one assumption at a time on host B. Pooled slots need 4.5–18.6% fewer reads and raise the bound at the host-bound budgets by 5–10%; starting MIN warm changes it by at most 1.4%; the median probe sample instead of the highest lowers it there by 8%. The GPU term moves most: at batch size 1 the all-in-VRAM runs reach only 52% (gpt-oss) and 61% (Qwen3) of the datasheet rate, and with the measured rate the bounds at 25% and above fall to 279 and 192 tok/s, 8–9% above the all-in-VRAM speed. In the running example (gpt-oss at 25%) the bound on host B is 430 tok/s (279 with the measured GPU rate) and the cache runs 109 tok/s. With every expert in its slots the cache matches stock llama.cpp ($0.997\times$ and $1.047\times$). The bound assumes an expert run on the CPU is read from DRAM (MIN’s shortest re-read distance is 0.5 GB of host traffic) and that computation runs at memory speed. The routing traces are each model’s own greedy text on the 30 prompts; replayed through a simulator of the deployed policy they reproduce the engine’s hit rates within 5.3 points.

Reaching the read time. The microbenchmark of Section 3 (`jobs/ec2/readsched.cu`) takes MIN’s misses per layer and step from `scripts/readsched.py` and runs no model. Usable cores minus two CPU threads read their share of host memory from an ordinary buffer, as the engine’s helpers do, and the copy engine copies the rest, 13.25 MB at a time, from pinned memory into GPU slots; both buffers rotate through 4 GiB so that no cache holds them. *Per layer* splits each layer’s experts, whole, between the two paths at the probe’s rates and waits for both before the next layer; *per token* sums the same split over the token and waits once; *link only* sends every read over PCIe, as admitting every miss does (MIN-1R sends only its admissions); *CPU only* reads everything on the CPU. Table 29 gives each machine. Waiting per layer instead of per token costs at most 1 point of the host term on the two AMD machines and 7 on the Intel machine, whose link is the slowest; there, reading everything on the CPU beats the per-layer split. The split ignores which reads are admissions; the link-aware bound sends every admission over the link and raises the bound by at most 1% at 11% (`scripts/linkaware.py`,

Table 29. Can the bound’s read time be reached (job 102)? MIN’s per-layer host reads of gpt-oss (first 4,000 steps of the AIME routing), replayed without the model through CPU threads and the copy engine. Each mode’s share of the bound’s host term (R^*S/B_{host} , ms per token in the column *Term*); *Model*: the per-layer mode predicted from the probe’s rates. Two repetitions of each mode agree within 0.9%.

Machine	Link/ CPU	C	Term (ms)	Split per layer	Split per token	Link only	CPU only	Model
Pd again	0.40	14	7.04	86%	93%	35%	92%	98%
Pd again	0.40	32	2.84	89%	93%	35%	92%	98%
9950X	1.02	14	9.70	96%	97%	95%	86%	94%
9950X	1.02	32	3.91	96%	97%	94%	86%	92%
9800X3D	0.55	14	9.56	95%	95%	53%	91%	95%
9800X3D	0.55	32	3.85	93%	94%	53%	91%	94%

Table 30. FETCH tables at 43.75% on Qwen3-30B-A3B (launch 1): entry n is how many of a layer’s n missed experts are copied over PCIe; the rest run on the CPU. Ratio to the fixed table, paired, 95% CI.

Table	$f(0..8)$	tok/s	vs fixed
Lite	0, 0, 1, 1, 1, 2, 2, 3, 3	109.0	1.057 [1.052, 1.062]
Model	0, 0, 1, 1, 2, 3, 3, 4, 5	108.0	1.048 [1.041, 1.055]
Model, fetch a single miss	0, 1, 1, 1, 2, 3, 3, 4, 5	104.1	1.010 [1.007, 1.012]
Fixed	0, 1, 1, 2, 3, 3, 4, 5, 6	103.1	1
None (CPU only)	0, . . . , 0	100.7	0.977 [0.970, 0.984]
All (fetch every miss)	0, 1, . . . , 8	98.7	0.957 [0.953, 0.962]

prereg/linkaware.json).

Choosing the split per machine. On host B, the model’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0% (launch 1). Table 30 shows why at one budget: a single missed expert should run on the CPU (about 155 μ s, hidden behind the GPU’s other experts) rather than be copied (about 225 μ s, on the critical path); that entry alone is worth 3.7%. Fetching everything, FreeToken’s offload strategy expressed in our engine, is the worst table, and running everything on the CPU the second worst. The model’s table was within 1% of the best of five alternatives on launch 1 and tied with it on launch 2. Given each run’s measured misses, the calibrated model of the supplement ranks the tables correctly except the model’s table against the lite one, which are within 1%, and overstates their differences about 1.8 $\times$.

Machines with half the link. On RTX 5090s whose links we measured at 27–30 GB/s instead of 53 (jobs 085, 085b and 088: gpt-oss next to a Core i9-14900K, Qwen3 next to a Ryzen 9 7900 in a PCIe 4.0 slot), the model’s tables fetch less (gpt-oss 0, 0, 0, 1, 1, Qwen3 0, 0, 1, 1, 2, 2, 3, 3, 4) and beat the fixed table by 21–34% on gpt-oss and 9–18% on Qwen3, about twice the gain on host B for Qwen3 and four to six times for gpt-oss: the per-machine split matters most where the link is slow. On the Ryzen, ours leads FreeToken by 1.04–1.08 $\times$; with the fixed table it would trail at 12.5%. On the Core i9 the first comparison (job 085) was confounded: FreeToken’s hybrid backend ran gpt-oss at 12–66 tok/s, below its own offload backend (25–86), because its CPU executor spread 23 threads across the 8 performance and 16 efficiency cores and its calibration then sent two thirds of the misses over the slow link, the same failure the supplement reports for llama.cpp; the 1.4–2.0 $\times$ lead that gave us measured a thread configuration. We reran it (job 088, a second rental of the same CPU model) with FreeToken’s executor on 8 threads pinned to the performance cores: its hybrid backend then runs

1.4–3.1 $\times$ faster than before (37.5, 62.1 and 92.5 tok/s; its calibration now copies 51.7% of misses), beating its own offload backend at 11 and 25% and tying it at 40%, and ours leads its better configuration by 1.58, 1.52 and 1.51 $\times$ (lowest interval bound 1.47). Restricted to the CPU alone, FreeToken’s 8-thread executor decodes 13.7 tok/s at every budget. llama.cpp on the same rental runs 40.0 tok/s with one thread per performance core (-9.4% from the model’s prediction) and 32.8 with all 24 threads (+25.9%): the hybrid-core failure is a thread-count choice, and the model holds once the choice is right. The model predicted llama.cpp on the Ryzen within +2.7%. A figure in the supplement puts every same-machine comparison on one axis.

Long outputs. Our lead does not come from the first 256 tokens. On gpt-oss, on ten held-out hard problems (AIME 2022, problems 11–15 of both exams) that neither system has finished reasoning about after 2,048 tokens (at most 1 of 10 reach an answer), ours runs 1.36 [1.34, 1.38], 1.32 [1.30, 1.34] and 1.19 [1.17, 1.21] times FreeToken at 11, 25 and 40% over all 2,048 tokens, and 1.37 [1.34, 1.39], 1.33 [1.31, 1.35] and 1.19 [1.17, 1.22] over tokens 257–2,048 (job 083, host A); both engines stream one event per token. On Qwen3, over tokens 257–2,048 of ten held-out MATH-500 problems, ours runs 1.043 [1.020, 1.070], 1.056 [1.036, 1.077] and 1.054 [1.025, 1.086] times FreeToken at 12.5, 25 and 43.75%. Both leads are larger than over the same runs’ first 256 tokens. Turning off FreeToken’s prefill overlap, which lends it cache slots during each prompt, changes its overall speed by 1% or less.

FreeToken’s own headline model. On FreeToken’s own headline model, Qwen3.6-35B-A3B in BF16 (256 experts of 6.3 MB per layer, hybrid linear attention), the two systems are close: ours runs 1.030 [1.023, 1.038] times FreeToken at 12.5% and ties at 25 and 37.5% (1.010 [0.998, 1.021], 1.003 [0.994, 1.013]), at 1.98–2.48 $\times$ llama.cpp (job 086, host A). FreeToken as shipped, with its automatic backend and cache size, runs this model at 93.8 tok/s on that machine, above the 77–83 tok/s its paper reports on an RTX 5090, so our setup does not handicap it; FreeToken’s fetch cap and thread count were left at their defaults, while our FETCH table was swept. Our lead is therefore model-dependent: largest with few large experts (gpt-oss, top-4 of 128), smallest with many small ones.

FreeToken tuned. On an RTX 5090 with a Ryzen 9 9950X (job 098, one launch, configurations in a shuffled order per cell) FreeToken ran in five settings per cell: hybrid with its calibrated fetch split (its default), hybrid with the fetch cap fixed at one and at two, hybrid with 8 CPU threads instead of one per physical core, and offload. Its best setting was the calibrated hybrid with 8 threads at three cells and offload at the other three (gpt-oss 25 and 40%, Qwen3 43.75%), at most 1.04 $\times$ its Table 3 setting; the fixed caps were never better than its own split by more than 0.3%. Ours over FreeToken’s best: 1.17, 1.17 and 1.09 on gpt-oss, 1.02, 1.04 and 0.98 on Qwen3, within 0.04 of host S’s ratios at every cell. With every expert on the CPU, stock llama.cpp decodes gpt-oss 1.13 $\times$ and Qwen3 0.99 $\times$ as fast as FreeToken’s CPU backend.

Other systems and outputs. Two newer llama.cpp-based systems, Pipelined Sharding (Ukarande et al., 2026) and leloch’s expert cache (leloch, 2026a), run gpt-oss-120b at 1.28–1.45 $\times$ stock llama.cpp; ours ran 1.42–2.34 $\times$ the better of them (job 077). KTransformers (Chen et al., 2025a) (kt-kernel 0.7.0, exact BF16 CPU experts, the same number of experts statically on the GPU) ran Qwen3 in BF16 at 0.98–1.01 $\times$ stock llama.cpp on an RTX 5090 with a Ryzen 9 9950X, and ours at 2.2–4.4 $\times$ it (job 078); both static placements read the same CPU expert bytes from the same DRAM. Its published speeds rely on AMX Xeons and INT4/INT8 CPU weights, which that machine lacks and which change the model, so this is not its best setting. Teacher-forced on fixed text (1536 steps per model) against stock llama.cpp with every weight in the VRAM of an RTX PRO 6000 (job 090), our cache at 25% has a mean KL divergence over the full vocabulary of 0.0019 nats on gpt-oss and 0.0005 on Qwen3 (99.9th percentile 0.121 and 0.059, maximum 0.21 and 0.13), picks the same next token on 98.5% and 99.3% of steps, and changes the loss by +0.16% and +0.19%. Stock llama.cpp’s own `--n-cpu-moe` placement diverges from the same reference by as much (mean KL 0.0017 and 0.0008, agreement 98.8% and 99.3%): the differences are those of running experts on the CPU (the helpers quantise activations to Q8_0 where the GPU path uses Q8_1), not of the cache. Task accuracy was not measured.

Prompts and batches. The cache serves single-token steps; prompts and batches above one run llama.cpp’s stock graph with every expert in host memory, and pay for the cache’s resident experts and its one-thread CPU side. In llama-batched-bench at 25% on host S (job 089, both models) our prefill at 512 and 2,048 tokens runs 79–83% of stock llama.cpp’s and our decode at 2 and 4 parallel sequences 77–79% of it (the tool feeds both engines the same random tokens).

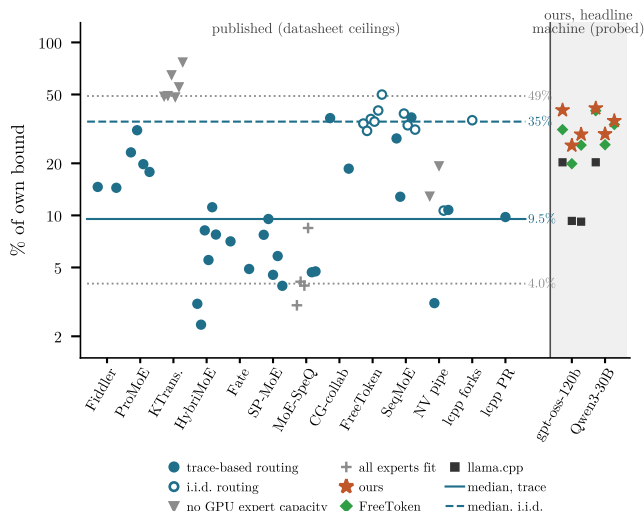


Figure 9. Published MoE-offloading measurements (52 rows from 13 sources) as a percentage of their own bound, computed from the paper’s hardware at datasheet rates and, where a routing trace of the model exists, the per-layer Belady optimum (filled); hollow points assume uniform routing, which overstates the optimum’s reads and so the percentage; the dotted lines are the class medians (no GPU expert capacity, 49.0%; every expert resident, 4.0%). Right: our cache, FreeToken and stock llama.cpp on host B against the probed per-layer bound of Table 3 (the pooled bound FreeToken’s and llama.cpp’s designs allow is 0–10% higher).

K THE AUDIT’S ROWS

Table 31 lists the 52 published measurements of Section 3 with their bounds. The sources are Fiddler (Kamahori et al., 2024), ProMoE (Song et al., 2024a), KTransformers (Chen et al., 2025a), HybriMoE (Zhong et al., 2025), Fate (Fang et al., 2025), SP-MoE (Chen et al., 2025b), MoE-SpeQ (Wang et al., 2025), the CPU–GPU collaborative inference of Huang et al. (2025), FreeToken (Yang et al., 2026), SeqMoE (Wang et al., 2026b), NVIDIA’s pipelined sharding (Ukarande et al., 2026), a llama.cpp fork (leloch, 2026a) and a llama.cpp pull request (csantiago78, 2026). The in-class subset of Section 3 (20 rows) is the trace-scored rows less the lossy (Fate) and speculative (SP-MoE, MoE-SpeQ) ones; over all 29 trace-scored rows the median is 9.5%. These systems mostly read on demand, so Eq. (2) would place them higher. Three caveats apply to every row: the ceilings are datasheet ones where ours are probed; the trace is the model’s own sampled text, sometimes of a sibling variant (an FP8 trace for a Q4 deployment); and systems whose cache is pooled across layers, whose placement pins whole layers, or which verify several tokens per pass are scored against the per-layer one-token bound, which is not theirs.

Table 31: The audit’s rows (Section 3): every published batch-1 measurement we scored, with the system’s stated hardware, its experts per layer resident on the GPU (C), the model’s experts and top- k , the published speed, the bound at datasheet-physical ceilings (GPU datasheet bandwidth, the top of the host-DRAM band, datasheet PCIe, no latencies; `scripts/audit.py`), how the optimum’s reads were obtained (*trace*: Belady’s MIN with bypass on an on-policy routing trace of the model; *uniform*: independent uniform routing, which overstates the reads and so the percentage; *no slots*: no GPU expert capacity, the CPU-only rate is the bound; *all fit*: every expert resident, nothing offloaded) and the percentage of the bound. Medians: trace 9.5% ($n=29$), uniform 34.8% (11), no slots 49.0% (8), all fit 4.0% (4), all rows 14.5% (52). Rows below 5% are more likely configuration mismatches with the bound’s assumptions (precision, dense layers on the CPU, end-to-end timing) than inefficiency, and are not adjudicated.

System	Model	GPU	C	E/k	tok/s	bound	reads	%
Fiddler (Kamahori et al., 2024)	Mixtral-8x7B	Quadro RTX 6000	1	8/2	2.5	17	trace	14.6
	Mixtral-8x7B	RTX 6000 Ada	3	8/2	8.3	57	trace	14.4
ProMoE (Song et al., 2024a)	DeepSeek-V2-Lite	RTX 4090	45	64/6	49.9	216	trace	23.1

continued

Where the Seconds Go

System	Model	GPU	C	E/k	tok/s	bound	reads	%
KTransformers (Chen et al., 2025a)	Qwen1.5-MoE-A2.7B	RTX 4090	46	60/4	67.2	216	trace	31.1
	Qwen2-57B-A14B	RTX 4090	42	64/8	26.9	136	trace	19.8
	Mixtral-8x7B	RTX 4090	6	8/2	27.0	152	trace	17.8
	DeepSeek-V3-0324	A100 40GB	0	256/8	5.9	12	no slots	48.8
	DeepSeek-V2.5-1210	A100 40GB	0	160/6	11.9	24	no slots	49.2
	Qwen2-57B-A14B	A100 40GB	0	64/8	22.9	35	no slots	64.9
	DeepSeek-V3-0324	RTX 4080	0	256/8	16.1	33	no slots	48.3
	DeepSeek-V2.5-1210	RTX 4080	0	160/6	17.6	32	no slots	55.3
	Qwen2-57B-A14B	RTX 4080	0	64/8	37.0	48	no slots	76.9
	DeepSeek-V2-Lite-Chat	RTX A6000	16	64/6	14.5	470	trace	3.1
HybriMoE (Zhong et al., 2025)	DeepSeek-V2-Lite-Chat	RTX A6000	48	64/6	15.3	655	trace	2.3
	Mixtral-8x7B-Instruct	RTX A6000	2	8/2	4.4	53	trace	8.2
	Mixtral-8x7B-Instruct	RTX A6000	6	8/2	7.3	133	trace	5.5
	Qwen2-57B-A14B-Instruct	RTX A6000	16	64/8	9.6	86	trace	11.1
	Qwen2-57B-A14B-Instruct	RTX A6000	48	64/8	9.0	116	trace	7.7
	Qwen1.5-MoE-A2.7B	RTX 3090	46	60/4	14.0	198	trace	7.1
	Qwen1.5-MoE-A2.7B	GTX 1080 Ti	15	60/4	3.7	76	trace	4.9
	Mixtral-8x7B	RTX 3090	1	8/2	1.2	16	trace	7.7
	Mixtral-8x7B	RTX 4090	1	8/2	1.3	13	trace	9.5
	Mixtral-8x7B	A100 40GB	3	8/2	1.5	32	trace	4.5
Fate (Fang et al., 2025)	Phi-3.5-MoE	RTX 4090	3	16/2	3.3	57	trace	5.8
	DeepSeek-V2-Lite	A100 40GB	7	64/6	5.6	142	trace	3.9
	DeepSeek-V2-Lite	A100 40GB	64	64/6	10.1	335	all fit	3.0
	DeepSeek-V2-Lite	A100 40GB	64	64/6	13.8	335	all fit	4.1
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	13.2	335	all fit	3.9
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	28.3	335	all fit	8.5
	Phi-3.5-MoE	A100 40GB	7	16/2	6.0	129	trace	4.7
	Phi-3.5-MoE	A100 40GB	7	16/2	6.1	129	trace	4.7
	Mixtral-8x7B	RTX 4090	1	8/2	4.8	13	trace	36.5
	Phi-3.5-MoE	RTX 4090	3	16/2	10.4	56	trace	18.6
SP-MoE (Chen et al., 2025b)	Qwen3.6-35B-A3B	RTX 5090	94	256/8	77.1	227	uniform	34.0
	DeepSeek-V4-Flash	RTX 5090	28	256/6	24.9	81	uniform	30.8
	Qwen3.6-35B-A3B	RTX 5090	104	256/8	73.8	205	uniform	36.1
	Qwen3.6-35B-A3B	RTX 4090	72	256/8	42.9	123	uniform	34.8
	Qwen3.6-35B-A3B	RTX 4060 Lap-top	60	256/8	39.3	97	uniform	40.4
	GLM-5.2	RTX PRO 6000 Blackwell	36	256/8	14.9	30	uniform	49.9
	Qwen3-30B-A3B-FP8	RTX 4090	57	128/8	104.1	374	trace	27.9
	Qwen3-30B-A3B-FP8	RTX 4090	19	128/8	47.9	374	trace	12.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	102	256/8	118.3	305	uniform	38.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	38	256/8	70.2	212	uniform	33.2
MoE-SpeQ (Wang et al., 2025)	GPT-OSS-120B-MXFP4	RTX PRO 6000 Blackwell	57	128/4	139.2	378	trace	36.8
	DeepSeek-V4-Flash	RTX PRO 6000 Blackwell	115	256/6	57.4	183	uniform	31.3
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
CPU-GPU collab. (Huang et al., 2025)	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
FreeToken (Yang et al., 2026)	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
SeqMoE (Wang et al., 2026b)	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
Pipelined shard-ing (Ukarande et al., 2026)	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9

continued

Where the Seconds Go

System	Model	GPU	C	E/k	tok/s	bound	reads	%
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	44	128/8	32.1	1032	trace	3.1
	Qwen3-235B-A22B-Instruct-2507	RTX 5090	0	128/8	7.7	40	no slots	19.3
	Qwen3-235B-A22B-Instruct-2507	RTX 5090	42	128/8	11.5	108	uniform	10.7
	Qwen3-30B-A3B-Instruct-2507	RTX 5070 Ti	107	128/8	54.9	511	trace	10.8
llama.cpp fork (leloch, 2026a)	Laguna S 2.1	RTX 5090	66	256/10	31.3	88	uniform	35.5
llama.cpp PR 27861 (csan-tiago78, 2026)	Qwen3-30B-A3B-Q4_K_M derivative	Radeon RX 7600 8GB	32	128/8	16.5	169	trace	9.8

L ARTIFACT DESCRIPTION

- **System.** A patch to llama.cpp commit 4da6337 (the expert cache, FETCH, the mailbox and the llama-ec-bench instrumentation: teacher-forced loss, logit dumps, routing lookahead records); built with CUDA 13.0 for compute capability 12.0.
- **Jobs.** One shell script per job; each installs FreeToken at commit 0d652e7, downloads the models from Hugging Face, runs the bandwidth probes, writes any on-machine prediction, runs the client, and stores raw rows (one JSON line per problem), server logs and engine counters. A small runner rents a Vast instance, streams a heartbeat, fetches the results and only then destroys the instance.
- **Analysis.** scripts/*_stats.py compute every interval from the raw rows; scripts/perlayer_model.py, speed_limit_v2.py, shapley_gap.py, policy_study.py, foresight_exact.py, batch_k_trace.py, fig_audit_sol.py and scorecard.py produce the model comparison, the bound, the accounting, the trace studies, the audit figure and the scorecard; scripts/wsg_tables.py and wsg_numbers2.py generate every table, figure and number in this paper from their outputs. The audit’s 52 published rows and the physical speed-of-light model they are scored against are data/published_measurements.json, data/audit/ and scripts/audit.py.
- **Cost.** The 124 rentals (74 distinct offers) of jobs 058–108 cost 87.2 US dollars at their hourly GPU prices, by the ledger in the repository; storage and download charges, which the ledger omits, add to this.